

Programmazione: breve guida di stile

Diagram for the computation by the Engine of the Numbers of Bernoulli. See Note G. (page 722 et seq.)

Number of Operations.	Names of Operations.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data												Working Variables.												Result Variables.			
						v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}	v_{11}	v_{12}	v_{13}	v_{14}	v_{15}	v_{16}	v_{17}	v_{18}	v_{19}	v_{20}	v_{21}	v_{22}	v_{23}	v_{24}	v_{25}	v_{26}	v_{27}	
						0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
						1	2	n																									
1	$\times$	$v_2 \times v_4$	v_6	v_6	$v_6 = v_2$	...	2	n	2n	2n																							
2	$-$	$v_4 - v_1$	v_4	v_4	$v_4 = v_2 - 1$	...	1	...	2n-1																								
3	$+$	$v_4 + v_1$	v_4	v_4	$v_4 = v_2 + 1$	...	1	...	2n+1																								
4	$+$	$v_4 + v_1$	v_6	v_6	$v_6 = v_4$	...			0	0																							
5	$+$	$v_6 + v_4$	v_6	v_6	$v_6 = v_4 + 1$	...	2	...																									
6	$-$	$v_6 - v_4$	v_6	v_6	$v_6 = v_4 - 1$	...																											
7	$-$	$v_6 - v_4$	v_6	v_6	$v_6 = v_4 - 1$	...	1	...	n																								
8	$+$	$v_2 + v_1$	v_2	v_2	$v_2 = v_1 + 1$	...	2	...																									
9	$+$	$v_6 + v_2$	v_6	v_6	$v_6 = v_2 + 1$	...																											
10	$\times$	$v_6 \times v_4$	v_6	v_6	$v_6 = v_4$	...																											
11	$+$	$v_6 + v_4$	v_6	v_6	$v_6 = v_4 + 1$	...																											
12	$-$	$v_6 - v_4$	v_6	v_6	$v_6 = v_4 - 1$	...	1	...																									
13	$-$	$v_6 - v_4$	v_6	v_6	$v_6 = v_4 - 1$	...	1	...																									
14	$+$	$v_4 + v_2$	v_2	v_2	$v_2 = v_1 + 1$	...	1	...																									
15	$+$	$v_6 + v_2$	v_6	v_6	$v_6 = v_2 + 1$	...																											
16	$\times$	$v_6 \times v_4$	v_6	v_6	$v_6 = v_4$	...																											
17	$-$	$v_6 - v_4$	v_6	v_6	$v_6 = v_4 - 1$	...	1	...																									
18	$+$	$v_4 + v_2$	v_2	v_2	$v_2 = v_1 + 1$	...	1	...																									
19	$+$	$v_6 + v_2$	v_6	v_6	$v_6 = v_2 + 1$	...																											
20	$\times$	$v_6 \times v_4$	v_6	v_6	$v_6 = v_4$	...																											
21	$\times$	$v_6 \times v_4$	v_6	v_6	$v_6 = v_4$	...																											
22	$+$	$v_6 + v_4$	v_6	v_6	$v_6 = v_4 + 1$	...																											
23	$-$	$v_6 - v_4$	v_6	v_6	$v_6 = v_4 - 1$	...	1	...																									
Here follows a repetition of Operations thirteen to twenty-three.																																	
24	$+$	$v_6 + v_4$	v_6	v_6	$v_6 = v_4 + 1$	...																											
25	$+$	$v_4 + v_2$	v_2	v_2	$v_2 = v_1 + 1$	...	1	...	n+1																								

Moreno Marzolla
Dipartimento di Informatica—Scienza e Ingegneria
Università di Bologna

Ultimo aggiornamento: 18 aprile 2025

Copyright 2022–2025 Moreno Marzolla www.moreno.marzolla.name



Quest'opera è rilasciata con licenza Creative Commons Attribuzione - Condividi allo stesso modo 4.0 Internazionale. Per leggere una copia della licenza visita il sito web <http://creativecommons.org/licenses/by-sa/4.0/> o spedisce una lettera a Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Indice

1 Forma.....	9
1.1 Indentazione.....	9
1.2 Spaziatura.....	10
1.3 Layout.....	12
1.4 Identificatori.....	13
1.5 Commenti.....	14
1.6 Costanti magiche.....	16
1.7 Rendere espliciti gli intenti.....	18
1.8 Priorità degli operatori.....	19
1.9 Lunghezza delle funzioni.....	20
2 Sostanza.....	23
2.1 Evitare ripetizioni.....	23
2.2 Semplificare le strutture condizionali.....	28
2.3 for o while?.....	30
2.4 break e continue.....	35
2.5 Array vs Tuple.....	37
2.6 Variabili globali.....	38
2.7 Ricorsione.....	39
2.8 Asserzioni.....	40
2.9 Invarianti di ciclo.....	41
2.10 Programmazione difensiva.....	43
2.11 Evitare soluzioni “astute”.....	45
2.12 Stack vs Heap.....	48
2.13 Macro.....	49
3 Esempi.....	51
3.1 I led lampeggianti.....	51
3.2 Controlla la data.....	53
3.3 sc.....	54
3.4 Conversione di un carattere numerico in un intero.....	56

Introduzione

Programmare è l'attività mediante la quale si istruisce un dispositivo di calcolo su come risolvere un problema o una classe di problemi. Per fare questo si utilizza un linguaggio di programmazione (C, C++, Java, Python, ...), la cui struttura è definita in modo rigido per essere “compreso” da un calcolatore elettronico.

Molte persone scrivono programmi pensando che siano destinati esclusivamente al compilatore (che li tradurrà in codice macchina) o all'interprete (che li eseguirà). Niente di più sbagliato: un programma è il mezzo con cui si esprime un'idea; programmare è quindi un atto creativo simile alla scrittura di una lettera. Come una lettera, un programma deve esprimere un concetto in modo chiaro; a differenza di una lettera, un programma deve essere scritto rispettando regole molto rigide, imposte dal linguaggio di programmazione, affinché venga interpretato o compilato correttamente.

Un programma potrà richiedere modifiche o estensioni successive, per aggiungere nuove funzionalità o correggere errori scoperti durante l'uso; per questo motivo è importante che l'intento del programma sia evidente, soprattutto nel caso in cui il codice debba essere modificato da una persona diversa da colui/colei che l'ha scritto. Un programma è quindi destinato sia alle macchine che alle persone, per cui è necessario adottare delle regole di stile: in molti casi, un programma verrà letto molte più volte di quante venga scritto.

Un programma, quindi, dovrebbe soddisfare alcuni requisiti spesso in conflitto tra loro:

- *Comprensibilità*: l'intento del programmatore deve risultare chiaro dal codice e dall'eventuale documentazione allegata (es., i commenti nel codice sorgente).
- *Mantenibilità*: deve essere facile aggiungere nuove funzionalità o correggere errori. Un programma dovrebbe essere strutturato in modo tale da consentire modifiche localizzate.
- *Efficienza*: un programma non deve richiedere una quantità eccessiva di risorse (tempo, spazio di memoria) per produrre il risultato.

Questa dispensa descrive alcune buone pratiche di programmazione che hanno lo scopo di migliorare la qualità dei programmi. Queste pratiche derivano da idee ampiamente condivise dalla comunità informatica, che sono già state messe per iscritto in molti libri (si vedano i riferimenti bibliografici in fondo a questo documento).

Le regole che presenteremo vanno intese come suggerimenti da cui è possibile derogare quando ci siano ragioni valide per farlo. Si diventa programmatori esperti quando si matura l'esperienza necessaria per capire quando derogare; fino ad allora consiglio di attenersi il più possibile alle regole.

In questa dispensa vengono usati frammenti di codice di esempio in linguaggio C o Java, per cui è necessario un minimo di familiarità con questi linguaggi. Tuttavia, la maggior parte delle regole sono valide a prescindere dal linguaggio di programmazione che si usa.

Chi desidera approfondire gli argomenti trattati può consultare il libro:

Steve McConnell, *Code Complete*, 2nd edition, Microsoft Press; 2004, ISBN 978-0735619678

(Personalmente ritengo l'acquisto di questo libro uno dei migliori investimenti che abbia fatto: non c'è una pagina da cui non abbia imparato qualcosa di utile!) Un altro libro consigliato è

Brian W. Kernighan and P. J. Plauger, *The Elements of Programming Style*, 2nd edition, McGraw-Hill, 1978, ISBN 0-07-034207-5

che sebbene molto datato (gli esempi di codice sono in FORTRAN e PL/I) è comunque rilevante anche oggi. Questa dispensa è fortemente ispirata al libro di Kernighan e Plauger, da cui vengono ripresi molti dei consigli.

È importante sottolineare che, pur se ampiamente condivise, le pratiche di programmazione descritte in questa dispensa non sono universali: molte aziende hanno le proprie, che possono essere in contrasto con questo scritto nel seguito. Inoltre, linguaggi di programmazione diversi hanno consuetudini diverse: ad esempio, nel linguaggio C si preferiscono identificatori brevi, mentre in Pascal o Java si usano identificatori più lunghi e descrittivi. È quindi necessario adattarsi al contesto in cui si sviluppa il codice.

La dispensa è divisa in due parti. Nella prima parte (“Forma”) vengono illustrate alcune regole per migliorare la forma dei programmi. Non è solo una questione estetica: un programma ordinato è più facile da comprendere e rende quindi più evidenti gli errori. La seconda parte (“Sostanza”) è dedicata alle regole per migliorare le funzionalità dei programmi, ad esempio per renderli più efficienti o robusti.

Sebbene questo documento sia rivolto principalmente a studenti e alle studentesse dei miei corsi, ritengo che possa essere utile anche a chi sta affrontando per la prima volta la programmazione, e magari anche a chi già lavora nel settore.

Cesena, aprile 2025

1 Forma

*Any fool can write code that a computer can understand.
Good programmers write code that humans can understand.*

(Martin Fowler e Kent Beck)

1.1 Indentazione

L'indentazione è la pratica di far rientrare le righe di codice sorgente in modo da evidenziarne la struttura a blocchi. Una indentazione appropriata aiuta a cogliere a colpo d'occhio la struttura di un programma. Come esempio, consideriamo una funzione `find_max()` che restituisce il valore massimo in un array non vuoto di lunghezza n . Mostriamo due versioni identiche tranne che per l'indentazione.

✗ Nessuna indentazione	✓ Indentazione corretta
<pre>int find_max(int a[], int n) { int i, m = a[0]; for (i = 1; i < n; i++) { if (a[i] > m) m = a[i]; } return m; }</pre>	<pre>int find_max(int a[], int n) { int i, m = a[0]; for (i = 1; i < n; i++) { if (a[i] > m) m = a[i]; } return m; }</pre>

Nella versione indentata correttamente (a destra) si nota subito la presenza del ciclo `for` e della struttura condizionale al suo interno; chi ha un minimo di pratica di programmazione riconosce questo schema e può intuire lo scopo della funzione. Codice non indentato (a sinistra) o indentato male non è solo difficile da leggere, ma può anche nascondere errori. Consideriamo l'esempio seguente in linguaggio C:

✗ Indentazione fuorviante	✓ Indentazione corretta
<pre>if (a > max) a = max; b = max - 1;</pre>	<pre>if (a > max) a = max; b = max - 1;</pre>

Nel codice a sinistra potrebbe sembrare che l'istruzione `b = max - 1` venga eseguita se `a > max`. In realtà non è così: il ramo "vero" non è racchiuso tra parentesi graffe, per cui in base al-

le regole del C è composto solo dalla riga successiva. A destra viene mostrato lo stesso codice indentato in modo corretto.

Il problema precedente è legato anche alla sintassi del linguaggio C, che consente di evitare l'uso di parentesi graffe nel caso in cui i blocchi siano composti da una singola istruzione. Personalmente preferisco utilizzare sempre le parentesi graffe, in modo che errori legati all'indentazione possano essere intercettati per tempo; un esempio abbastanza famoso di un errore critico legato alla mancanza di parentesi graffe è il cosiddetto “*goto fail bug*” introdotto da Apple nella sua versione della libreria SSL/TLS:

```
static OSStatus
SSLVerifySignedServerKeyExchange(SSLContext *ctx,
                                bool isRsa,
                                SSLBuffer signedParams,
                                uint8_t *signature,
                                UInt16 signatureLen)
{
    OSStatus err;
    /* omissis */

    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
    goto fail; /* ERRORE */
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;
    /* omissis */

fail:
    SSLFreeBuffer(&signedHashes);
    SSLFreeBuffer(&hashCtx);
    return err;
}
```

Per errore è stata duplicata la riga `goto fail`; tuttavia, visto che non sono presenti le parentesi graffe, la seconda istruzione `goto fail` viene eseguita comunque, anche se la condizione dell’`if` precedente è falsa. In altre parole, nessuna delle istruzioni comprese tra il secondo `goto fail` e l’etichetta `fail` viene mai eseguita.

Alcuni linguaggi di programmazione, come Python, sfruttano l’indentazione per definire la struttura a blocchi di un programma. In Python non esistono parentesi graffe o altri simboli per raggruppare istruzioni in blocchi, ma il raggruppamento deriva esclusivamente dall’indentazione.

Esistono diversi stili di indentazione. Alcuni (es., lo “stile GNU”) posizionano le parentesi graffe su righe a sé stanti, mentre altri (es., lo “stile K&R”, da Kernighan & Ritchie, gli autori del manuale “The C Programming Language”) favoriscono un layout più compatto.

✓ Indentazione stile “GNU”	✓ Indentazione stile “K&R”
<pre> int power(int x, int n) { int result; if (n < 0) { result = 0; } else { result = 1; while (n > 0) { result *= x; n--; } } return result; } </pre>	<pre> int power(int x, int n) { int result; if (n < 0) { result = 0; } else { result = 1; while (n > 0) { result *= x; n--; } } return result; } </pre>

Personalmente adotto l’indentazione K&R perché rende il codice compatto e quindi sfrutta meglio lo spazio a disposizione. Nello stile K&R le parentesi graffe di apertura di un blocco vengono messe sulla stessa riga della parola chiave `if`, `else`, `while`, `for`; le parentesi di chiusura vanno su una riga a sé stante, tranne nel caso del ramo vero di un `if` seguito da `else`. Le parentesi graffe di apertura e chiusura del corpo di una funzione vanno invece su righe a sé stanti.

Al di là delle preferenze personali, è importante scegliere uno stile di indentazione e usarlo in modo consistente. Quasi tutti gli ambienti di sviluppo consentono di indentare il codice in modo automatico, per cui non è accettabile scrivere programmi in modo sciatto.

☞ Indentare il codice in modo consistente.

1.2 Spaziatura

Un programma fitto e privo di spazi è confuso e difficile da leggere. Ad esempio, consideriamo il seguente frammento di codice Java:

✗ Difficile da leggere	✓ Spaziatura adeguata
<pre> int i,j; while(i<j){//Un commento int m=(i+j)/2;//Un altro commento if(v[i]<k) j=m-1;//Terzo commento else if(v[i]>k) i=m+1;//Fine } </pre>	<pre> int i, j; while (i < j) { // Un commento int m = (i+j)/2; // Un altro commento if (v[i] < k) j = m-1; // Terzo commento else if (v[i] > k) i = m+1; // Fine } </pre>

La versione di destra è più leggibile, ed è stata ottenuta aggiungendo spazi:

- attorno alle parentesi () nelle espressioni condizionali del `while` e degli `if`;

- attorno ad alcuni operatori (<, > e =);
- prima delle parentesi graffe aperte {
- prima e dopo il delimitatore di inizio dei commenti //

È possibile aggiungere spazi in altri punti (es., attorno agli operatori – e + nelle espressioni). Come per gli stili di indentazione, non ci sono regole fisse per inserire gli spazi; ciascuno dovrà quindi trovare il proprio stile. Come sempre, in assenza di vincoli stringenti (es., dettati dal proprio datore di lavoro), si usi buon senso.

Un discorso analogo vale per la spaziatura verticale: in molti casi ricevo codice con righe vuote posizionate a caso o in modo non consistente, che rendono difficile la lettura e sfruttano male lo spazio verticale.

 Usare spazi e “a capo” in modo consistente per favorire la leggibilità del codice.

1.3 Layout

I primi videoterminali erano in grado di visualizzare 25 righe per 80 colonne di testo; queste dimensioni erano imposte principalmente dai limiti tecnologici dei tubi catodici e da altre convenzioni risalenti agli albori dell'informatica. Oggi questi limiti sono superati: i monitor hanno una risoluzione elevata che, combinata con il formato dell'immagine 16:9, fornisce ampio spazio in orizzontale. Ritengo tuttavia che non sia una buona idea sfruttare questo spazio per scrivere righe di codice lunghe, per almeno due motivi:

- È molto scomodo leggere righe di testo lunghe, come ci si può rendere conto da questa pagina (volutamente stampata in modalità “landscape”). Non è un caso che i libri siano solitamente più alti che larghi, e i quotidiani usino colonne di testo molto più strette della larghezza della pagina;
- Non è detto che chi riceve il codice abbia una risoluzione orizzontale sufficiente per visualizzare le righe senza che vengano mandate a capo o troncate. Anche nel caso in cui si disponga di una risoluzione orizzontale sufficiente, c'è chi ingrandisce i font per leggere meglio, oppure chi preferisce tenere diverse finestre affiancate; questo riduce lo spazio orizzontale a disposizione.

Molti editor di testo usano di default una larghezza di 80 caratteri, oppure mostrano un riferimento (es., una linea verticale) in corrispondenza della colonna 80. Suggerisco di fare il possibile per mantenere le righe di codice e i commenti entro gli 80 caratteri.

☞ Mantenere la lunghezza delle righe di codice e di commento al di sotto di 80 caratteri.

Non tutti condividono questo suggerimento. Linus Torvalds, l'autore del sistema operativo Linux, ritiene che spezzare righe di codice sia fonte di potenziali problemi, per cui ritiene accettabile scrivere righe di codice lunghe. Le sue argomentazioni, che possono essere lette all'indirizzo <http://lkml.iu.edu/hypermail/linux/kernel/2005.3/08168.html>, sono in parte condivisibili ammesso che si riesca a tollerare l'atteggiamento da “bulletto del quartierino” con cui le espone (è il suo marchio di fabbrica).

1.4 Identificatori

Gli *identificatori* sono nomi a cui vengono associate delle entità del linguaggio di programmazione; le entità possono essere variabili, costanti, tipi di dato, funzioni e altro. Ogni linguaggio definisce le proprie regole per la costruzione di identificatori. Ad esempio, in C/C++ un identificatore può contenere esclusivamente lettere minuscole, maiuscole, cifre numeriche e il simbolo `_`, e non può iniziare con una cifra numerica. In Java è possibile usare anche il simbolo `$`, e certi tipi di identificatori devono iniziare con una lettera maiuscola (es., i nomi di classi). In alcuni linguaggi esiste un limite alla lunghezza di un identificatore. Identificatori non validi vengono di solito rifiutati dal compilatore o dall'interprete.

Ogni linguaggio ha le proprie consuetudini sui nomi degli identificatori. In C si preferiscono identificatori brevi, anche composti da una singola lettera. Nei linguaggi della famiglia del Pascal (Pascal, Ada, Delphi, ecc.) si preferiscono nomi lunghi. Trattandosi di consuetudini e non di regole formali, ciascuno è libero di adottare il proprio stile per la scelta dei nomi, anche se suggerisco di adeguarsi alle consuetudini del linguaggio.

 Attenersi alle consuetudini del linguaggio per la scelta dei nomi degli identificatori.

Il nome di un identificatore dovrebbe soddisfare due requisiti contrastanti:

- Essere abbastanza breve, per ridurre la probabilità di commettere errori di scrittura.
- Suggestire il significato dell'oggetto a cui si riferisce.

Un identificatore breve non è sempre significativo, mentre un identificatore significativo non è sempre breve. In generale può essere difficile trovare un compromesso tra queste due esigenze.

Nella tabella che segue vengono riportati alcuni suggerimenti che fanno riferimento alle consuetudini del linguaggio C. Questi suggerimenti vanno considerati come tali, e non devono essere intesi come regole rigide.

	✗ No	✓ Sì
Indici di cicli	indice1 indice2 indice3	i j k
Temperatura in gradi C	x	temp temperatura
Dimensioni di una matrice	dim1 dim2 a b	n m righe colonne r c
Somma di una sequenza di valori	prod tmp val ¹	sum somma s
Albero dei cammini minimi (<i>minimum spanning tree</i>)	a tree	mst minSpanningTree min_spanning_tree
Funzione che crea una struttura “grafo”	f() crea()	creaGrafo() crea_grafo() nuovoGrafo() nuovo_grafo() make_graph() mkGraph()

Per migliorare la leggibilità degli identificatori lunghi suggerisco di usare il carattere `_` come separatore (es., `min_spanning_tree`) anziché le maiuscole (es., `minSpanningTree`). Il motivo è che siamo già abituati a leggere parole separate da spazi, per cui la soluzione che usa il carattere `_` è in qualche modo più naturale. Tuttavia, trattandosi di preferenze personali ciascuno è libero di regolar-si diversamente.

👉 Usare nomi appropriati per gli identificatori.

1.5 Commenti

Tutti i linguaggi di programmazione consentono di inserire commenti nel codice sorgente. I commenti possono essere usati per diversi scopi:

- per documentare l’uso di funzioni, classi, metodi;
- per descrivere parti del codice che potrebbero risultare di difficile comprensione;
- per specificare metadati relativi al programma (es., autore, data dell’ultima modifica, licenza, ecc.);
- per specificare direttive particolari da passare a preprocessori o estensioni del compilatore.

Ogni funzione non banale deve essere preceduta da un commento che specifichi:

- cosa calcola (*postcondizione*);
- qual è il significato dei parametri (se presenti);

¹ Sì, mi sono stati consegnati programmi in cui una sommatoria veniva chiamata in questi modi.

- quali vincoli ciascun parametro deve soddisfare (es., se è un puntatore, deve essere diverso da NULL? deve avere particolari valori, es. maggiore o uguale a zero?). Tali vincoli sono detti *precondizioni*.

Seguono alcuni esempi.

✗ Commento inutile (troppo generico)

```
/* Seleziona un elemento. */
int select(int v[], int n, int k);
```

✓ Commento appropriato

```
/**
 * Restituisce la posizione (indice) del valore di rango k in v[]; il
 * valore di rango k è quello che occuperebbe la k-esima posizione se v[]
 * fosse ordinato; questo problema prende il nome di "problema della selezione"
 *
 * Input:
 * - v[] array non vuoto di interi;
 * - n lunghezza di v[]; si deve avere n>0;
 * - k rango dell'elemento da selezionare (k=0 indica che si vuole il minimo,
 *   k=n-1 indica che si vuole il massimo); si deve avere 0 <= k <= n-1
 * Output:
 * - posizione (indice) in v[] dell'elemento di rango k; si garantisce che
 *   il risultato sia sempre compreso tra 0 e n-1, inclusi.
 * Nota: questa funzione permuta il contenuto di v[]
 */
int select(int v[], int n, int k);
```

I commenti sono una parte importante di qualsiasi programma, e vanno scritti con la dovuta cura. Si evitino commenti poco comprensibili o, peggio, sgrammaticati. perché mettono in cattiva luce il resto del codice.

 Commentare i punti critici del codice in modo chiaro, completo e grammaticalmente corretto.

Un errore comune è quello di commentare il codice riga per riga. Ciò ha come unico risultato quello di infastidire chi lo legge e appesantire inutilmente il codice.

✗ Commenti inutili

```
v = v + 1; /* incrementa v */
if (v > 10) { /* se v è maggiore di 10 */
    v = 0; /* setta v a zero */
}
Prim(G, v); /* esegui l'algoritmo di Prim su G */
```

✓ Commenti appropriati

```
/* Individua la posizione i del primo valore negativo
nell'array a[] di lunghezza n >= 0; al termine si ha
i == n se non esiste alcun valore negativo */
int i = 0;
while (i < n && a[i] >= 0) {
    i++;
}
```

Invece di parafrasare il codice, usare i commenti per descrivere le funzionalità di una porzione di codice o l'algoritmo impiegato per risolvere il problema, nel caso in cui queste informazioni non siano facilmente comprensibili guardando il codice.

👉 Mai parafrasare il codice riga per riga.

I commenti possono essere utili per descrivere il significato di variabili o costanti, nei casi in cui i nomi non siano auto-esplicativi. Non è sempre necessario descrivere tutte le variabili usate, ma è buona norma farlo almeno per quelle più importanti o il cui significato non sia immediato.

✓ Commenti appropriati

```
int boardDim;           /* dimensione (lato) della scacchiera */
float g_cor, g_max;      /* guadagno corrente e massimo */
double min_cost;        /* costo del cammino di costo minimo */
```

Un problema frequente dei commenti è che talvolta non sono allineati con il codice: spesso capita di fare una modifica al codice dimenticandosi di modificare il commento associato. Ad esempio, il commento alla funzione `check()` seguente indica che essa restituisce un valore, ma il codice indica che il tipo di ritorno è `void`. Chi ha ragione?

✗ Codice e commenti non concordano

```
/* Restituisce true (nonzero) se l'array è ordinato in senso crescente */
void check(int *v, int n);
```

👉 Assicurarsi che codice e commenti concordino.

Nell'esempio precedente è presente anche una scelta poco felice del nome della funzione. Supponendo che `check()` ritorni un intero: cosa rappresenta un risultato non-zero (*true*)? che l'array è ordinato oppure no? È possibile eliminare l'ambiguità scegliendo un nome diverso:

✓ Codice e commenti concordano; il nome della funzione rende il commento superfluo

```
/* Restituisce true (nonzero) se l'array è ordinato in senso crescente */
int is_sorted(int *v, int n);
```

In questo caso è evidente che se la funzione ritorna *true* significa che l'array è ordinato.

1.6 Costanti magiche

Capita spesso di usare costanti numeriche i cui valori hanno un significato particolare. Ad esempio, supponiamo di modellare la propagazione di molecole di gas su una superficie piana rappresentata da una matrice in due dimensioni. Ogni cella della matrice può essere vuota, oppure contenere una molecola di gas, oppure contenere un ostacolo. È possibile rappresentare ciascuno stato con un valore intero, ad esempio 0, 1, 2. Tuttavia, così facendo chi legge il codice potrebbe non capire immediatamente a quale stato corrisponde ciascun valore: lo zero rappresenta una cella vuota o una cella che contiene un ostacolo?

I letterali numerici² diversi da 0 e 1 che compaiono in un sorgente sono detti “costanti magiche”. Ogni ambiguità può essere risolta introducendo degli identificatori con nomi significativi:

✗ Cosa rappresentano i valori 0, 1, 2?	✓ Codice non ambiguo
<pre>const int a = v[i][j]; const int b = v[i-1][j]; const int c = v[i][j-1]; const int d = v[i-1][j-1]; if (((a == 0) != (b == 0)) && ((c == 1) != (d == 1))) (a == 2) (b == 2) (c == 2) (d == 2)) { ... }</pre>	<pre>enum {EMPTY, GAS, WALL}; const int a = v[i][j]; const int b = v[i-1][j]; const int c = v[i][j-1]; const int d = v[i-1][j-1]; if (((a == EMPTY) != (b == EMPTY)) && ((c == GAS) != (d == GAS))) (a == WALL) (b == WALL) (c == WALL) (d == WALL)) { ... }</pre>

Nella versione a destra si usano le costanti EMPTY, GAS e WALL, definite tramite un tipo enumerativo del linguaggio C. Il tipo enumerativo consente di definire simboli a cui assegnare valori interi costanti; nel caso in cui i valori non siano assegnati esplicitamente, vengono usati gli interi 0, 1, Di conseguenza, la dichiarazione del tipo enumerativo definisce tre costanti EMPTY = 0, GAS = 1 e WALL = 2.

Un linguaggio di programmazione potrebbe mettere a disposizione diversi meccanismi per definire costanti. Ad esempio, in C sarebbe stato possibile definire costanti globali (*named constants*) con le dichiarazioni:

```
const int EMPTY = 0;
const int GAS = 1;
const int WALL = 2;
```

oppure usando simboli del preprocessore:

```
#define EMPTY 0
#define GAS 1
#define WALL 2
```

Ciascuna delle soluzioni precedenti (tipi enumerativi, costanti o simboli del preprocessore) ha pregi e difetti, ma risponde ugualmente bene al requisito di sostituire le costanti numeriche con nomi significativi.

Un altro caso in cui spesso si abusa di “costanti magiche” è nella gestione di codici ASCII. Nel linguaggio C l'espressione letterale 'x' (singoli apici, da non confondere con "x") ha valore intero e corrisponde al codice ASCII del carattere x. Questo fatto può essere utilizzato per rendere più comprensibile il codice che manipola caratteri.

Supponiamo di voler scrivere una funzione `maiuscolo(x)` che dato un carattere x restituisce la versione maiuscola di x se x è un carattere alfabetico, altrimenti restituisce x stesso. Ad esempio,

² Un *letterale numerico* è la rappresentazione di un numero. Esempi di letterali numerici in C sono -13, 0, 3.14e2. In C si possono usare anche letterali di tipo carattere (es., 'a') oppure stringa (es., "ciao"). Si presti attenzione che c'è differenza tra *costanti* e *letterali*: una costante è un nome a cui è associato un valore che non può essere cambiato da programma. Un letterale è una sequenza di simboli che rappresenta direttamente un valore.

`maiuscolo('a') == 'A'`, mentre `maiuscolo('?') == '?'`. Per scrivere la funzione sfruttiamo il fatto che, nella codifica ASCII, i caratteri alfabetici 'a'...'z' e 'A'...'Z' hanno codici in sequenza. La funzione `maiuscolo()` usa il tipo `int` anziché `char` per compatibilità con la funzione di libreria `toupper()` definita in `<ctype.h>` che realizza la stessa operazione.

✗ No: usa “costanti magiche”	✓ Ok
<pre>int maiuscolo(int x) { if (x >= 97 && x <= 122) return (x - 97) + 65; else return x; }</pre>	<pre>int maiuscolo(int x) { if (x >= 'a' && x <= 'z') return (x - 'a') + 'A'; else return x; }</pre>

La versione che usa codici numerici è difficile da capire, perché non è evidente a quali caratteri corrispondano le “costanti magiche” 97, 122, 65. Sfruttando quanto detto sopra, possiamo sostituire i numeri con letterali di tipo carattere, ottenendo una funzione molto più leggibile: non dobbiamo ricordarci che il codice ASCII di 'a' è 97, dato che 97 è il valore dell'espressione 'a'.

Come ulteriore esempio, supponiamo di dover leggere una sequenza di caratteri e stampare solo quelli corrispondenti alle cifre numeriche '0', ... '9'. Anche in questo caso sfruttiamo il fatto che i codici ASCII dei caratteri '0', ... '9' hanno valori consecutivi; non serve ricordarsi che il codice ASCII di '0' è 48, in quanto il compilatore può ricavare autonomamente questa informazione:

✗ No: usa “costanti magiche”	✓ Ok
<pre>int c; while ((c = getchar()) != EOF) { if (c >= 48 && c <= 57) { printf("%c", c); } }</pre>	<pre>int c; while ((c = getchar()) != EOF) { if (c >= '0' && c <= '9') { printf("%c", c); } }</pre>

Si noti che `c` deve avere tipo `int` e non `char`; la funzione `getchar()` restituisce il codice ASCII del carattere letto da standard input, oppure il valore EOF se l'input è terminato. Normalmente EOF è definito come -1, quindi `getchar()` restituisce un valore nell'insieme {-1, 0, ... 255} che non può essere rappresentato da un `char`.

☞ Usare espressioni letterali, simboli o “named constants” al posto di costanti magiche.

I valori 0 e 1 sono talmente comuni da non richiedere costanti simboliche che li definiscano. Ci possono essere alcune situazioni in cui non c'è vantaggio nel sostituire letterali con costanti, per cui ogni situazione va valutata caso per caso.

1.7 Rendere espliciti gli intenti

Può essere utile assegnare dei nomi ai valori di certe espressioni per chiarirne l'intento. Come semplice esempio, consideriamo una funzione `reverse()` il cui scopo è di invertire il contenuto di un array (il primo elemento diventa l'ultimo, il secondo diventa il penultimo e così via).

✗ Intento non evidente	✓ Intento evidente
<pre>void reverse(int v[], int n) { for (i = 0; i < n/2; i++) { const int tmp = v[i]; v[i] = v[n-1-i]; v[n-1-i] = tmp; } }</pre>	<pre>void swap(int *a, int *b) { ... } void reverse(int v[], int n) { for (i = 0; i < n/2; i++) { const int opp = n-1-i; swap(&v[i], &v[opp]); } }</pre>

La versione a sinistra è corretta e lo scopo della funzione è evidente dal nome. La funzione è anche molto corta, per cui nella pratica **non vale la pena ristrutturarla**. Tuttavia, ragionando un po' di più, il significato dell'espressione `n-1-i` potrebbe non essere del tutto ovvio (è l'indice dell'elemento in posizione simmetrica rispetto a `v[i]`). La versione di destra rende evidente l'intento del codice grazie a due modifiche:

- la creazione di una funzione ausiliaria `swap()`, il cui nome suggerisce lo scopo (scambiare i valori puntati dagli argomenti);
- la definizione di una costante `opp` (*opposite*) per rappresentare l'indice dell'elemento opposto all'indice `i`, cioè l'elemento corrispondente all'indice `i` nella seconda metà dell'array.

Il corpo del ciclo nella nuova funzione `reverse()` potrebbe ora essere letto come: “scambia `v[i]` con l'elemento nella parte opposta”.

 Rendere evidente l'intento del codice.

1.8 Priorità degli operatori

Ogni linguaggio di programmazione stabilisce la priorità degli operatori aritmetici e logici usati nelle espressioni. Ad esempio, in linguaggio C l'espressione

```
a == 2 || c == 3
```

viene interpretata come se fosse scritta

```
(a == 2) || (c == 3)
```

perché l'operatore di confronto (`==`) ha precedenza maggiore rispetto all'“or logico” (`||`). Occorre però prestare attenzione, perché le priorità degli operatori potrebbero essere definite in modo contro-intuitivo. Ad esempio, supponiamo di voler confrontare il valore di una variabile *status* con una maschera binaria composta utilizzando l'operatore “or bit a bit”. Questo frammento di codice non è corretto

```
if (status == BIT_RD | BIT_WR | BIT_EXCL) { ... } /* ERRATO */
```

perché l'operatore “or bit a bit” (`|`) ha priorità *minore* rispetto all'operatore di confronto, per cui la condizione viene valutata come se fosse scritta:

```
if ((status == BIT_RD) | BIT_WR | BIT_EXCL) { ... }
```

Per ottenere il comportamento corretto servono quindi le parentesi:

```
if (status == (BIT_RD | BIT_WR | BIT_EXCL)) { ... }
```

Ogni linguaggio di programmazione riserva sorprese relativamente alla priorità degli operatori. È sconsigliabile tenere a memoria la tabella della priorità; conviene invece usare le parentesi quando non si è sicuri.

☞ Usare le parentesi per evitare errori legati alla priorità degli operatori.

1.9 Lunghezza delle funzioni

Funzione o metodi troppo lunghi, cioè con troppe righe di codice, sono difficili da comprendere; non è un caso che uno dei principi di base della programmazione sia la *decomposizione* di problemi complessi in problemi più semplici. È molto più facile capire cosa fa un blocco di poche righe di codice, piuttosto che una sequenza di centinaia di istruzioni. Una funzione troppo lunga è un chiaro segnale dell'opportunità di decomporla in funzioni più semplici.

Non esistono regole meccaniche per decidere quando e come decomporre una funzione. Un indicatore è la lunghezza in righe di codice: funzioni più lunghe di 50-60 righe potrebbero necessitare di essere decomposte, sebbene ci possano essere eccezioni. Un altro indicatore è il livello di annidamento dei blocchi, inteso come il numero di blocchi inseriti uno nell'altro (es., `if` all'interno di `for` all'interno di `while` all'interno di altri `if`...). Codice che presenti più di 3-4 livelli di annidamento è difficile da comprendere e può nascondere errori insidiosi. Questi problemi possono essere risolti spostando in funzioni separate il codice che si trova nei blocchi più interni.

☞ Decomporre funzioni troppo lunghe o complesse.

Come corollario a quanto detto sopra, la funzione `main()` del linguaggio C o il metodo `main()` di Java dovrebbero limitarsi a delegare l'esecuzione del programma ad altre funzioni. Tranne che nei casi più semplici, scrivere tutto il codice all'interno del `main()` è una pessima pratica di programmazione.

☞ Tranne nei casi più semplici, non scrivere l'intero programma nella funzione `main()`.

Come esempio consideriamo la funzione `floyd_warshall()` che implementa l'algoritmo di Floyd e Warshall per il calcolo dei cammini minimi da singola sorgente (u dettagli dell'algoritmo non sono importanti per capire l'esempio).

✘ Prima della decomposizione

```
int floyd_warshall(const Graph *g,
                  double **d, int **p)
{
    int u, v, k;
    const int n = graph_n_nodes(g);
    for (u = 0; u < n; u++) {
        for (v = 0; v < n; v++) {
            d[u][v] = (u==v ? 0.0 : HUGE_VAL);
            p[u][v] = -1;
        }
    }
    for (u = 0; u < n; u++) {
        const Edge *e;
        for (e = graph_adj(g, u);
             e != NULL; e = e->next) {
            d[e->src][e->dst] = e->weight;
            p[e->src][e->dst] = e->src;
        }
    }
    for (k = 0; k < n; k++) {
        for (u = 0; u < n; u++) {
            for (v = 0; v < n; v++) {
                if (d[u][k]+d[k][v]<d[u][v]) {
                    d[u][v] = d[u][k] + d[k][v];
                    p[u][v] = p[k][v];
                }
            }
        }
    }
    for (u = 0; u < n; u++) {
        if ( d[u][u] < 0.0 ) return 1;
    }
    return 0;
}
```

✔ Dopo la decomposizione

```
void init_fw(const Graph *g,
            double **d, int **p)
{
    int u, v;
    const int n = graph_n_nodes(g);
    for (u = 0; u < n; u++) {
        for (v = 0; v < n; v++) {
            d[u][v] = (u==v ? 0.0 : HUGE_VAL);
            p[u][v] = -1;
        }
    }
    for (u=0; u<n; u++) {
        const Edge *e;
        for (e = graph_adj(g, u);
             e != NULL; e = e->next) {
            d[e->src][e->dst] = e->weight;
            p[e->src][e->dst] = e->src;
        }
    }
}

void relax(double **d, int **p,
          int k, int n)
{
    int u, v;
    for (u = 0; u < n; u++) {
        for (v = 0; v < n; v++) {
            if (d[u][k]+d[k][v]<d[u][v]) {
                d[u][v] = d[u][k] + d[k][v];
                p[u][v] = p[k][v];
            }
        }
    }
}

int neg_cycle(double **d, int n)
{
    int u;
    for (u = 0; u < n; u++) {
        if ( d[u][u] < 0.0 ) return 1;
    }
    return 0;
}

int floyd_warshall(const Graph *g,
                  double **d, int **p)
{
    int k;
    const int n = graph_n_nodes(g);
    init_fw(g, d, p);
    for (k=0; k<n; k++) {
        relax(d, p, k, n);
    }
    return neg_cycle(d, n);
}
```

L'algoritmo si compone di tre parti: nella prima vengono inizializzati la matrice delle distanze $d[]$ e l'array dei predecessori $p[]$; nella seconda si eseguono n passi di rilassamento, dove n è il numero di nodi del grafo; nell'ultima parte si verifica la presenza di cicli negativi, dato che in presenza di cicli negativi potrebbero non esistere cammini di lunghezza minima.

La versione non fattorizzata di `floyd_warshall()` (a sinistra) è già accettabile così com'è, dato che ha una lunghezza ragionevole ed è comprensibile almeno per coloro che conoscono l'algoritmo. Non è quindi indispensabile decomporla; tuttavia, se lo si fa si ottiene una versione un po' più leggibile, dato che risultano evidenti le tre fasi di cui è costituita. Tuttavia, l'applicazione eccessiva del principio di decomposizione può portare più problemi che benefici: troppe chiamate a funzioni obbligano chi legge a saltare da una parte all'altra del sorgente alla ricerca della definizione delle funzioni invocate. Occorre quindi valutare di volta in volta cosa conviene fare.

Per decidere quando una funzione è “troppo complessa” è possibile aiutarsi con delle metriche di *complessità strutturale*. Queste metriche non hanno nulla a che vedere con la complessità asintotica degli algoritmi; piuttosto, misurano quanto un frammento di codice sorgente è “ingarbugliato”. Una di queste metriche è la *complessità ciclomatica* proposta da Thomas K. McCabe nel 1976, e misura il numero di cammini indipendenti nel diagramma di flusso. Intuitivamente, la complessità ciclomatica aumenta all'aumentare del numero di strutture condizionali e iterative presenti nel codice.

La complessità ciclomatica è strettamente correlata alla validazione e test del codice. Infatti, per testare almeno una volta ogni possibile flusso di esecuzione è necessario un numero di casi di test maggiore o uguale al numero di cammini indipendenti nel diagramma di flusso (il test e la validazione del codice sono argomenti complessi che non vengono approfonditi qui). Di conseguenza, un valore elevato della complessità ciclomatica indica che è necessario un numero elevato di casi di test per attraversare ogni cammino del grafo di flusso.

Esistono software in grado di calcolare varie metriche di complessità strutturale, come `pmccabe` (<https://gitlab.com/pmccabe/pmccabe>). Per esaminare il contenuto del file `floyd-warshall.c` si usa il comando:

```
$ pmccabe -v floyd-warshall.c
```

```
Modified McCabe Cyclomatic Complexity
|   Traditional McCabe Cyclomatic Complexity
|   |   # Statements in function
|   |   |   First line of function
|   |   |   |   # lines in function
|   |   |   |   |   filename(definition line number):function
|   |   |   |   |   |
12   12   40   149   50   floyd-warshall.c(149): floyd_warshall
3    3    6    202   15   floyd-warshall.c(202): print_path
3    3    15   220   18   floyd-warshall.c(220): print_dist
7    7    38   239   45   floyd-warshall.c(239): main
```

Le colonne di interesse sono le prime due, che indicano due varianti della complessità ciclomatica di McCabe, e la quinta che indica la lunghezza di ciascuna funzione. Le funzioni con complessità ciclomatica superiore a 10-15 devono essere esaminate con attenzione perché potrebbero richiedere una decomposizione.

2 Sostanza

Beware of bugs in the above code; I have only proved it correct, not tried it.

(Donald Knuth)

2.1 Evitare ripetizioni

Vanno evitate le ripetizioni di codice, perché rendono il programma più lungo e complesso da capire e favoriscono errori insidiosi; ad esempio, se la parte ripetuta deve essere in seguito modificato, c'è il rischio di dimenticarsi di cambiare tutte le copie. Un indizio sulla presenza di ripetizioni si ha quando si effettuano “copia e incolla” durante la scrittura del codice.

Esistono diversi modi per ridurre le ripetizioni. In molti casi si può dichiarare la parte ripetuta in una funzione, mentre in altri casi potrebbe essere utile definire la parte ripetuta all'interno di cicli. Ad esempio, supponiamo di dover realizzare una funzione `foo(in, out, n)` che riempie una matrice `out[][]` di dimensioni $n \times n$ a partire da una matrice `in[][]` delle stesse dimensioni. Il valore di ciascun elemento non sul bordo `out[i][j]` è la somma dei valori nelle otto celle adiacenti a `in[i][j]` a cui viene applicata una funzione `bar()`. In altre parole, si calcola la sommatoria della funzione `bar()` applicata agli elementi dell'intorno di dimensioni 3×3 centrato in `in[i][j]`, escluso il valore centrale `in[i][j]`.

La soluzione mostrata a sinistra realizza in modo diretto quanto sopra usando codice ripetuto. Nella versione a destra le ripetizioni sono eliminate mediante la definizione di una funzione di supporto `nbor()` e l'uso di cicli.

✖ Codice ripetuto	✔ Ok
<pre> void foo(int **in, int **out, int n) { int i, j; for (i = 1; i < n-1; i++) { for (j = 1; j < n-1; j++) { out[i][j] = bar(in[i-1][j-1]); out[i][j] += bar(in[i-1][j]); out[i][j] += bar(in[i-1][j+1]); out[i][j] += bar(in[i][j-1]); out[i][j] += bar(in[i][j+1]); out[i][j] += bar(in[i+1][j-1]); out[i][j] += bar(in[i+1][j]); out[i][j] += bar(in[i+1][j+1]); } } } </pre>	<pre> int nbor(int **in, int i, int j) { int s, t, result = 0; for (s = -1; s <= 1; s++) { for (t = -1; t <= 1; t++) { if (s t) result += bar(in[i+s][j+t]); } } return result; } void foo(int **in, int **out, int n) { int i, j; for (i = 1; i < n-1; i++) { for (j = 1; j < n-1; j++) { out[i][j] = nbor(in, i, j); } } } </pre>

Osserviamo che nella versione a sinistra sono presenti otto righe quasi identiche, in cui l'unica cosa che cambia sono gli indici nella matrice `in[][]`. Il codice è tedioso da modificare, ad esempio nel caso in cui venisse richiesto di usare un intorno di dimensioni maggiori, oppure di applicare una funzione diversa da `bar()`, dato che occorrerebbe modificare tutte le otto occorrenze della chiamata di `bar()`.

Nella versione di destra spostiamo il calcolo dei nuovi valori di `out[i][j]` in una funzione `nbor()`; la seconda consiste nell'uso di due cicli annidati per iterare sulle celle adiacenti a `in[i][j]`. Sebbene il programma diventi più lungo, è anche più comprensibile e più facile da modificare. La dimensione dell'intorno è determinata dagli estremi dei cicli `for` nella funzione `nbor()`; nel caso in cui si debba applicare una funzione diversa da `bar()` ai valori `in[i][j]` prima di accumularli, è sufficiente modificare un unico punto del codice.

👁 Limitare al minimo le ripetizioni.

Come ulteriore esempio, si vuole scrivere una funzione per calcolare l'importo delle tasse dovute su un reddito R . Le tasse corrispondono ad una percentuale t dell'intero reddito, che dipende da R secondo la tabella seguente:

Reddito (R)	Tasse (t)
$0 \leq R \leq 10000$	0,00%
$10000 < R \leq 15000$	5,00%
$15000 < R \leq 21000$	7,00%
$21000 < R \leq 34000$	8,00%
$34000 < R \leq 49500$	8,50%
$49500 < R \leq 62300$	10,00%
$62300 < R$	12,75%

Possiamo calcolare t traducendo la tabella in una catena di condizioni `if ... else`; il programma che si ottiene è piuttosto ripetitivo, e diventa ingestibile all'aumentare del numero di scaglioni.

✗ Codice ripetitivo	✓ Meno comprensibile, ma più compatto
<pre>double t, tasse; if (R <= 10000) { t = 0.0; } else if (R <= 15000) { t = 5.0; } else if (R <= 21000) { t = 7.0; } else if (R <= 34000) { t = 8.0; } else if (R <= 49500) { t = 8.5; } else if (R <= 62300) { t = 10.00; } else { t = 12.75; } tasse = R * t / 100;</pre>	<pre>/* s[i] è il limite inferiore dello scaglione i */ const double s[] = {0, 10000, 15000, 21000, 34000, 49500, 62300}; const double t[] = {0.0, 5.0, 7.0, 8.0, 8.5, 10.0, 12.75}; const int N = sizeof(s)/sizeof(s[0]); double t, tasse; int i = N-1; while ((i > 0) && (s[i] <= R)) i--; tasse = R * t[i] / 100;</pre>

È possibile semplificare il programma spostando la logica dal codice ai dati, che in questo caso è costituita da una coppia di array paralleli $s[]$ e $t[]$: $s[i]$ rappresenta il limite inferiore dello scaglione i , e $t[i]$ il corrispondente tasso di interesse. Si può determinare lo scaglione corretto (e quindi la percentuale di imposte) effettuando una ricerca lineare nell'array $s[]$ partendo dalla fine.

Confrontando le due versioni, si vede che il nuovo codice non è necessariamente migliore della soluzione diretta: la catena di condizioni, sebbene noiosa da scrivere, è più intuitiva da capire. La soluzione basata su array paralleli è più compatta se gli scaglioni sono molti, ma richiede commenti al codice per aiutare chi legge a capire cosa si sta facendo.

☞ Spostare la logica dal codice ai dati, quando è opportuno farlo.

Quando si decide di fattorizzare il codice, non sono considerate accettabili le soluzioni che consistono nello spostare i blocchi di codice ripetuto in altrettante funzioni diverse. Consideriamo la funzione seguente che accetta in input tre matrici $d[][]$ e $p[][]$ di dimensioni $r \times c$, e una matrice $w[][]$

di dimensioni $r \times 4$, e modifica ogni elemento di $d[][]$ e $p[][]$ tenendo conto del valore dei quattro elementi adiacenti:

```
enum {NORTH, SOUTH, EAST, WEST};

void f(float **d, /* dimensioni r x c */
    float **w, /* dimensioni r x 4 */
    int **p, /* dimensioni r x c */
    int r, int c) {
    int i, j;
    for (i=1; i<r-1; i++) {
        for (j=1; j<c-1; j++) {
            if (d[i][j] > d[i-1][j] + w[i][NORTH]) {
                d[i][j] = d[i-1][j] + w[i][NORTH];
                p[i][j] = NORTH;
            }
            if (d[i][j] > d[i+1][j] + w[i][SOUTH]) {
                d[i][j] = d[i+1][j] + w[i][SOUTH];
                p[i][j] = SOUTH;
            }
            if (d[i][j] > d[i][j-1] + w[i][EAST]) {
                d[i][j] = d[i][j-1] + w[i][EAST];
                p[i][j] = EAST;
            }
            if (d[i][j] > d[i][j+1] + w[i][WEST]) {
                d[i][j] = d[i][j+1] + w[i][WEST];
                p[i][j] = WEST;
            }
        }
    }
}
```

Si vede immediatamente che i quattro **if** sono praticamente identici. Tuttavia, sarebbe sbagliato definire quattro funzioni per ciascuno dei quattro blocchi ripetuti:

```
/* Soluzione alternativa non adeguata */
enum {NORTH, SOUTH, EAST, WEST};

void relax_NORTH(float **d, const float **w, int **p, int i, int j) {
    if (d[i][j] > d[i-1][j] + w[i][NORTH]) {
        d[i][j] = d[i-1][j] + w[i][NORTH];
        p[i][j] = NORTH;
    }
}
void relax_SOUTH( ... ) { idem }
void relax_EAST( ... ) { idem }
void relax_WEST( ... ) { idem }

void f(float **d, const float **w, int **p, int r, int c) {
    int i, j;
    for (i=1; i<r-1; i++) {
        for (j=1; j<c-1; j++) {
            relax_NORTH(d, w, p, i, j);
            relax_SOUTH(d, w, p, i, j);
            relax_EAST(d, w, p, i, j);
            relax_WEST(d, w, p, i, j);
        }
    }
}
```

```
}
```

In questo modo il numero di righe di codice non diminuisce perché si sono semplicemente spostati quattro blocchi di codice quasi identico in altrettante funzioni quasi identiche.

Una soluzione migliore consiste nel definire una *singola* funzione `relax()`, in cui la direzione (NORTH, SOUTH, eccetera) è passata come parametro. Occorre fare attenzione al fatto che, in base alla direzione, occorre accedere ad elementi diversi delle matrici `d[][]` e `w[][]`. Un modo per farlo è come segue:

```
enum {NORTH, SOUTH, EAST, WEST};

void relax(float **d, const float **w, int **p, int i, int j, int dir) {
    int i1 = i, j1 = j;
    switch (dir) {
        case NORTH: i1--; break;
        case SOUTH: i1++; break;
        case EAST: j1++; break;
        case WEST: j1--; break;
    }
    /* (i1, j1) sono le coordinate della cella adiacente a (i, j) lungo la direzione dir */
    if (d[i][j] > d[i1][j1] + w[i][dir]) {
        d[i][j] = d[i1][j1] + w[i][dir];
        p[i][j] = dir;
    }
}

void f(float **d, const float **w, int **p, int r, int c) {
    int i, j;
    for (i=1; i<r-1; i++) {
        for (j=1; j<c-1; j++) {
            relax(d, w, p, i, j, NORTH);
            relax(d, w, p, i, j, SOUTH);
            relax(d, w, p, i, j, EAST);
            relax(d, w, p, i, j, WEST);
        }
    }
}
```

Nella funzione `relax()` vengono calcolate le coordinate $(i1, j1)$ dell'elemento adiacente a (i, j) in base alla direzione `dir`: se `dir = NORTH`, $(i1, j1)$ deve essere l'elemento della matrice che si trova sopra a (i, j) ; se `dir = EAST` deve essere l'elemento che si trova a sinistra, e così via. Il nuovo codice è meno ridondante rispetto alla versione iniziale e può essere accettato già in questa forma. È tuttavia possibile effettuare un ulteriore passo spostando la logica nei dati come visto nell'esempio precedente.

Riscriviamo la funzione `relax()` nel modo seguente:

```
void relax(float **d, const float **w, int **p, int i, int j, int dir) {
    static const int di[] = {-1, 1, 0, 0}; /* NORTH, SOUTH, EAST, WEST */
    static const int dj[] = {0, 0, 1, -1};
    const int i1 = i + di[dir];
    const int j1 = j + dj[dir];
    if (d[i][j] > d[i1][j1] + w[i][dir]) {
        d[i][j] = d[i1][j1] + w[i][dir];
    }
}
```

```

        p[i][j] = dir;
    }
}

```

Questa versione fa uso di due array paralleli *di[]* e *dj[]* che vengono utilizzati per ottenere le coordinate (*i1*, *j1*) della cella che si trova, rispettivamente, a nord, sud, est e ovest rispetto a (*i*, *j*). L'espressione *di[dir]* indica quanto bisogna aggiungere (o togliere) alla coordinata *i* per ottenere la riga in cui si trova la cella adiacente a (*i*, *j*) lungo la direzione *dir*; analogamente, *dj[dir]* indica quanto bisogna aggiungere (o togliere) alla coordinata *j* per ottenere la colonna in cui si trova la cella adiacente lungo la direzione *dir*. La direzione è codificata dalle costanti enumerative NORTH, SOUTH, EAST, WEST, che hanno valore rispettivamente 0, 1, 2, 3 in base alle regole del linguaggio C. Di conseguenza, possiamo usare la direzione per indicizzare *di[]* e *dj[]* come mostrato.

Non è scontato che l'ultima versione presentata sia da preferire alla precedente: sicuramente è quella più elegante e compatta, ma ciò che si guadagna in compattezza si perde in leggibilità. Ciascuno dovrà quindi trovare un compromesso in base alle proprie esigenze e capacità.

2.2 Semplificare le strutture condizionali

Le strutture condizionali (*if-then-else*) vanno scritte in una delle due forme seguenti:

<code>if (Cond) Strue;</code>	<code>if (Cond) Strue; else Sfalse;</code>
-------------------------------------	--

dove *Cond* è una espressione, e *Strue*, *Sfalse* sono blocchi *non vuoti* di codice (eventualmente racchiusi tra parentesi graffe se composti da più di una istruzione). Si presti attenzione al fatto che devono essere non vuoti: ciò implica che i blocchi condizionali non dovrebbero mai avere il ramo “vero” vuoto.

✗ No: ramo “vero” vuoto	✓ Ok
<code>if (a < b) { /* vuoto */ } else { bar(); }</code>	<code>if (a >= b) { bar(); }</code>

Se si usano strutture condizionali annidate (cioè posizionate una dentro l'altra), è importante assicurarsi che le condizioni non siano ridondanti. Nell'esempio seguente, la condizione `if (a >= b)` è superflua, perché sappiamo già che nel ramo “falso” della condizione `if (a < b)` vale `a >= b`.

✖ Condizione superflua	✔ Ok
<pre>if (a < b) { foo(); } else { if (a >= b) { /* superflua */ bar(); } }</pre>	<pre>if (a < b) { foo(); } else { bar(); }</pre>

Nell'esempio seguente mostriamo un caso in cui non è presente ridondanza in senso stretto, cioè non c'è codice inutile, ma è possibile semplificare il programma rendendolo più compatto e chiaro. I quattro `if` annidati possono essere espressi con una singola condizione ottenuta mediante l'`and` logico di quelle di partenza.

✖ No	✔ Sì
<pre>if (A) { if (B) { if (C) { if (D) { blah(); } } } }</pre>	<pre>if (A && B && C && D) { blah(); }</pre>

Tale semplificazione non è sempre possibile. Ad esempio, i due frammenti di codice seguenti non sono equivalenti:

Versione 1	Versione 2 non equivalente
<pre>if (A) { if (B) { foo(); } } else { bar(); }</pre>	<pre>if (A && B) { foo(); } else { bar(); }</pre>

Per rendercene conto, esaminiamo tutte le combinazioni dei valori di *A* e *B*, e per ciascuna di esse determiniamo quale funzione viene invocata.

A	B	Funzione invocata dalla Versione 1	Funzione invocata dalla Versione 2
vero	vero	foo()	foo()
vero	falso	–	bar()
falso	vero	bar()	bar()
falso	falso	bar()	bar()

Osserviamo che se A è vera e B è falsa, la versione 1 non esegue alcuna istruzione, mentre la versione 2 invoca la funzione `bar()`.

Un altro caso di struttura che spesso può essere semplificata si ha quando si intende scrivere una espressione che restituisce “vero” o “falso” in base ad una determinata condizione. Ad esempio, supponiamo di voler scrivere una funzione `is_even_positive(n)` che restituisce “vero” se n è pari e positivo, zero altrimenti. La soluzione diretta con un `if-else` esplicito è sicuramente corretta, ma in linguaggio C si tende a preferire la versione più compatta di destra.

✓ Ok	✓ Meglio
<pre>int is_even_positive(int n) { if (((n % 2) == 0) && (n > 0)) { return 1; } else { return 0; } }</pre>	<pre>int is_even_positive(int n) { return ((n % 2) == 0) && (n > 0); }</pre>

Ricordiamo che in linguaggio C una espressione logica ha valore 0 se è falsa, 1 se è vera. Di conseguenza, nel caso in cui la condizione sia sufficientemente semplice, è possibile usare l'espressione come valore di ritorno, senza necessità di una struttura condizionale esplicita.

👉 Semplificare le strutture condizionali quando possibile.

2.3 for o while?

Le strutture `for` e `while` nei linguaggi C/C++ e Java sono interscambiabili, nel senso che un ciclo espresso mediante una delle due si può riscrivere in modo equivalente con l'altra. Esempio:

Usando “for”	Usando “while”
<pre>int i; for (i = 0; i < 100; i = i + 5) { blah(i); }</pre>	<pre>int i = 0; while (i < 100) { blah(i); i = i + 5; }</pre>

Ricordando che le strutture generali dei cicli `for` e `while` sono:

<pre>for (iniz; test; aggiornamento) { corpo }</pre>	<pre>while (test) { corpo }</pre>
--	---------------------------------------

Nonostante l'equivalenza, esistono delle convenzioni a cui è bene attenersi. Usare `for` o `while` dipende dal tipo di ciclo:

- Un *ciclo enumerativo* è governato da un contatore che spesso viene incrementato/decrementato di una quantità prefissata ad ogni iterazione; il numero di iterazioni risulta quindi prevedibile. Per questo genere di cicli si usa preferibilmente **for**.
- In un *ciclo a valutazione ripetuta* il numero di iterazioni dipende da una condizione complessa che viene valutata ogni volta; di conseguenza, il numero di iterazioni non è prevedibile. Per questo genere di cicli si usa preferibilmente **while**.
- Un *ciclo iteratore* ha come scopo quello di visitare tutti gli elementi di una struttura dati (es., una lista). Per questo genere di cicli si usa preferibilmente **for**; alcuni linguaggi come C++ e Java offrono una sintassi specifica.
- Un *ciclo infinito*, che non termina mai. Questo genere di cicli si trova spesso nella programmazione di microcontrollori e sistemi embedded in cui il dispositivo deve eseguire continuamente la stessa sequenza di istruzioni. Per questo genere di cicli si usa preferibilmente **while**; in C/C++ di solito si scrive **while(1) { ... }**, mentre in Java si scrive **while(true) { ... }**. È comunque piuttosto diffusa, e quindi perfettamente accettabile, la rappresentazione di un ciclo infinito usando l'idioma **for(;;) { ... }**

Alcuni esempi:

Cicli enumerativi	Cicli a valutazione ripetuta	Cicli iteratori
<pre>for (i=0; i<100; i+=4) { blah(i); }</pre>	<pre>/* Esempio C */ File *f = fopen("test.in", "r"); while (!feof(f)) { process_item(f); }</pre>	<pre>/* Esempio C */ typedef struct List { int val; struct List *next; } List; List *L = ..., *node; for (node = L; node != NULL; node = node->next) { blah(node); }</pre>
<pre>for (i=1; i<100; i=i*2) { blah(i); }</pre>	<pre>// Esempio Java File f = new File("test.in"); Scanner s = new Scanner(f); while (s.hasNextInt()) { int val = s.nextInt(); process(val); }</pre>	<pre>// Esempio Java LinkedList<Integer> L = ...; for (Integer item : L) { process(item); }</pre>

Possiamo definire alcune regole di tipo generale per decidere quando usare i due tipi di cicli:

Usare **for** se:

- Sono presenti tutte e tre le componenti (inizializzazione, test, aggiornamento), e
- inizializzazione, test e aggiornamento fanno riferimento alla stessa variabile, e
- la variabile che viene modificata nel blocco “aggiornamento” non viene modificata nel corpo del ciclo.

Usare **while** se:

- Il ciclo è “a valutazione ripetuta”, oppure
- la variabile utilizzata nel test vengono aggiornate in modo non uniforme all’interno del corpo del ciclo.

Il ciclo **for** si usa preferibilmente specificando tutte e tre le componenti (inizializzazione, test, aggiornamento), come nel codice a destra:

✗ No	✓ Si
<pre>i = 0; for (; i < 10; i++) { blah(i); }</pre>	<pre>for (i = 0; i < 10; i++) { blah(i); }</pre>
<pre>i = 0; for (; i < 10 ;) { blah(i); i++; }</pre>	
<pre>i = 0; for (; ;) { if (i >= 10) break; blah(i); i++; }</pre>	

In generale, un ciclo che itera su tutti gli interi compresi tra 0 e $n - 1$ viene scritto in C o Java nella seguente *forma idiomatica*:

```
for (i = 0; i < n; i++) {
    /* ... */
}
```

I programmatori esperti colgono a colpo d'occhio il significato del ciclo guardandone la struttura. Ogni linguaggio di programmazione ha le proprie forme idiomatiche, cioè degli schemi che vengono sistematicamente applicati per realizzare certe computazioni. Conviene seguire le forme idiomatiche per facilitare la comprensione del codice.

🔗 **Attenersi alle forme idiomatiche del linguaggio di programmazione.**

Come esempi di uso di **for** e **while** descriviamo due funzioni che operano su un array $v[]$ di lunghezza n . La funzione `esiste_dup()` restituisce *true* se e solo se esiste un valore in $v[]$ che compare almeno due volte. La funzione `binsearch()` cerca la posizione di una occorrenza di un valore dato usando l'algoritmo di ricerca binaria.

✔ Preferibile “for”	✔ Preferibile “while”
<i>/* Dato un array v[] di lunghezza n, restituisce “vero” se esiste un valore in v[] che compare almeno due volte, “falso” altrimenti. */</i> <pre> int esiste_dup(int v[], int n) { int i, j; for (i = 0; i < n-1; i++) { for (j = i+1; j < n; j++) { if (v[i] == v[j]) return 1; } } return 0; } </pre>	<i>/* Cerca una occorrenza di key nell'array v[] di lunghezza n; l'array deve essere ordinato in senso non decrescente. Restituisce l'indice dell'occorrenza trovata, oppure -1 se la chiave non è presente. */</i> <pre> int binsearch(int v[], int n, int key) { int i = 0; j = n-1; while (i <= j) { const int m = (i+j) / 2; if (v[m] < key) { i = m+1; } else if (v[m] > key) { j = m-1; } else { return m; } } return -1; /* chiave non presente */ } </pre>

Nel caso della ricerca di duplicati, la soluzione proposta può essere codificata usando due cicli `for`, in quanto i tre requisiti indicati sopra sono soddisfatti. Nel caso della ricerca binaria, le variabili che compaiono nel test del ciclo vengono aggiornate in modo non uniforme (in certi casi si aggiorna *i*, in altri si aggiorna *j*); ciò suggerisce di usare un ciclo `while`.

☞ Scegliere tra `for` e `while` in modo ragionato.

Come detto sopra, è consuetudine che all'interno del corpo di un ciclo `for` non si modifichi la variabile che viene aggiornata nell'istestazione del ciclo. Violare questa consuetudine confonde chi legge e può introdurre errori insidiosi.

Si consideri il problema seguente: è data una stringa *s[]* rappresentata in C mediante un array di caratteri terminato da `'\0'`. Vogliamo esaminare la stringa da sinistra verso destra, stampando il valore intero corrispondente alle sottostringhe massimali di cifre. Ad esempio, se *s* = “ab132ccd3ef21”, vogliamo isolare stampare i valori interi 132, 3 e 21. Consideriamo le due soluzioni seguenti; quella di sinistra è errata.

✘ Errato	✔ Corretto
<pre> for (i = 0; s[i]; i++) { if (isdigit(s[i])) { int v = 0; while (isdigit(s[i])) { v = v*10 + (s[i] - '0'); i++; /* attenzione */ } printf("%d\n", v); } } </pre>	<pre> i = 0; while (s[i]) { if (isdigit(s[i])) { int s = 0; while (isdigit(s[i])) { s = s*10 + (s[i] - '0'); i++; } printf("%d\n", s); } } </pre>

Nella soluzione di sinistra si modifica il valore della variabile *i* sia nell'intestazione del costrutto `for`, sia nel corpo del ciclo. Questo introduce un errore piuttosto insidioso: al termine del ciclo `while` interno, `s[i]` è il carattere immediatamente successivo all'ultima cifra letta. Tuttavia, la variabile *i* viene nuovamente incrementata al termine di ogni iterazione del `for` esterno: questo causa un accesso ad un elemento successivo all'ultimo se la stringa termina con una cifra, ad es., `s = "123a34"`. Per verificarlo è sufficiente inserire una istruzione `assert(i <= strlen(s))` nel codice a sinistra subito prima dell'`if`; parleremo della macro `assert()` nella Sezione 2.10.

L'errore non si verifica nella versione di destra; inoltre, la presenza di un `while` suggerisce la possibilità che all'interno del suo corpo il valore di *i* possa venire modificato, come effettivamente avviene.

☞ Non modificare la variabile indice di un ciclo “for” nel corpo del ciclo.

Una importante applicazione della regola precedente riguarda l'iterazione su una struttura dati bi-dimensionale, come ad esempio una matrice. L'idioma standard consiste nell'utilizzare due cicli annidati; vanno quindi evitate soluzioni del tutto inappropriate che usano un solo ciclo, come quella riportata qui sotto a sinistra. Si assume che `a[i][j]` sia una matrice con *R* righe e *C* colonne.

✗ No	✓ Si
<pre> for (i = 0, j = 0; i < R; j++) { if (j == C) { j = 0; i++; } if (i == R) break; do_something(a[i][j]); } </pre>	<pre> for (i = 0; i < R; i++) { for (j = 0; j < C; j++) { do_something(a[i][j]); } } </pre>

La versione a sinistra presenta molti problemi:

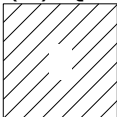
- Il test del ciclo si effettua su una variabile diversa da quella che viene incrementata nell'intestazione; in base a quanto detto in precedenza, ciò suggerisce che `for` non sia il costrutto appropriato.
- Entrambe le variabili del ciclo vengono modificate all'interno del corpo.
- Si deve usare una seconda condizione per evitare che il corpo del ciclo prenda in considerazione la riga di indice *R* (che non esiste in quanto gli indici validi in questo caso vanno da 0 a *R* - 1 inclusi).
- Si fa uso di una istruzione `break` (Sezione 2.4).
- L'intento del codice non è chiaro (Sezione 1.7).

La versione di sinistra è opaca, fragile ed inefficiente, e va evitata nella maggior parte dei casi.

☞ Iterare su matrici in modo esplicito usando cicli annidati.

2.4 break e continue

I costrutti `break` e `continue` vanno usati con parsimonia: dovrebbero essere l'eccezione più che la regola. L'istruzione `break` è problematica perché introduce una violazione del principio della programmazione strutturata, la quale richiede che ogni blocco abbia un singolo punto di ingresso e un singolo punto di uscita. Il punto di uscita “naturale” di un costrutto iterativo (`for` o `while`) è la condizione del ciclo che diventa falsa. Ad esempio, consideriamo il ciclo

```
while (P) {  
      
}
```

dove P è una espressione logica, e Q è un blocco di istruzioni. Supponendo che il ciclo prima o poi termini, quale proprietà vale al termine del ciclo, cioè subito dopo la parentesi graffa chiusa? Nel caso in cui siano rispettate le regole della programmazione strutturata, il ciclo termina se e solo se P diventa falsa, indipendentemente dalle istruzioni presenti in Q ; possiamo quindi ignorare il corpo del ciclo e concludere che, al termine, l'espressione P risulti sempre falsa.

Ad esempio, nel caso in cui il ciclo seguente termini:

```
int i = ...;  
while (i < 42) {  
    printf("i=%d\n", i);  
    i += 3;  
}
```

si avrà che $i \geq 42$, qualunque sia il valore iniziale della variabile i (occorre prestare attenzione al fatto che, in generale, non è sempre garantito che qualunque ciclo termini; si veda la sezione 2.10).

Se il blocco Q contiene uno o più `break`, allora quanto sopra non è necessariamente vero, dato che le proprietà che valgono al termine del ciclo dipendono dal contenuto di Q . Ad esempio, nel seguente frammento di codice:

```
int i = ...;  
while (i < 42) {  
    printf("i=%d\n", i);  
    if (i % 4 == 0)  
        break;  
    i += 3;  
}
```

è possibile uscire dal ciclo sia quando la condizione $i < 42$ diventa falsa, sia quando $i \% 4 == 0$ diventa vera. Si vede quindi come la presenza di `break` complichino l'analisi di un ciclo perché obblighi a tenere conto delle istruzioni presenti nel suo corpo.

In molti casi si può evitare l'uso di `break` ristrutturando la condizione di terminazione del ciclo. Ad esempio, supponiamo ad esempio di volere cercare la posizione (indice) del primo valore negativo in un array $v[]$ di lunghezza n . Una soluzione diretta sfrutta un ciclo enumerativo per scorrere l'array, e forzare l'uscita con `break` al primo valore negativo. È però possibile evitare il `break` con una condizione leggermente più complessa, come mostrato nel codice a destra.

✗ No	✓ Sì
<pre> for (i = 0; i < n; i++) { if (v[i] < 0) break; } if (i >= n) { Nessun valore negativo } else { v[i] è il primo valore negativo } </pre>	<pre> for (i=0; (i<n) && (v[i] >= 0); i++) { /* corpo vuoto */ } if (i >= n) { Nessun valore negativo } else { v[i] è il primo valore negativo } </pre>

Nella versione prima dell'istruzione `break` è possibile osservare che il ciclo termina quando la condizione `(i<n) && (v[i] >= 0)` diventa falsa, cioè se la sua negazione

$$!((i < n) \ \&\& \ (v[i] \geq 0)) \equiv (i \geq n) \ || \ (v[i] < 0)^3$$

è vera. In altre parole, il ciclo termina se “ $i \geq n$ ” oppure “ $v[i] < 0$ ” che è ciò che deve calcolare.

Lo schema precedente è abbastanza frequente, per cui è utile fornire una regola generale. Supponiamo di voler iterare un ciclo enumerativo fino a quando una certa condizione *cond* diventa vera. Supponendo che *test* e *cond* non abbiano “effetti collaterali”, cioè non modifichino variabili, si può evitare il `break` modificando la condizione di terminazione del ciclo come mostrato di seguito:

✗ No	✓ Sì
<pre> for (init; test; incr) { if (cond) break; else foo(); } </pre>	<pre> for (init; test && !cond; incr) { foo(); } </pre>
	<pre> /* soluzione alternativa */ init; while (test && !cond) { foo(); incr; } </pre>

Vanno evitate soluzioni “pigre” come la seguente:

✗ No
<pre> int stop = 0; for (i = 0; (i < n) && !stop; i++) { if (v[i] < 0) stop = 1; } if (stop) { Nessun valore negativo } else { v[i] è il primo valore negativo } </pre>

³ Il simbolo $\equiv$ si legge “equivale a”

Nel code precedente si è introdotta una variabile “stop” per forzare l'esecuzione del ciclo. Sebbene tecnicamente non sia presente l'istruzione `break`, l'analisi del ciclo usando invarianti di ciclo non è affatto semplificato: al termine del ciclo `for` sappiamo che

`!((i < n) && !stop) ≡ (i >= n) || stop`

è vera, ma questo da solo non ci consente di dimostrare che il ciclo calcola quello che deve: per fare questo occorre esaminare il corpo, che è esattamente quello che si vorrebbe evitare.

Come esempio di abuso del costrutto `continue`, consideriamo una funzione che, data una matrice `m[][]` di caratteri di dimensioni 3×3 , restituisce 0 se tutti i caratteri nella matrice sono uguali ad 'A', 0 altrimenti. La versione di sinistra si basa sull'idea che, se il valore `m[i][j]` è uguale ad 'A', allora si prosegue con il controllo, altrimenti si interrompono i cicli restituendo 1. In questo caso il `continue` può essere facilmente eliminato invertendo la condizione dell'`if`, come mostrato nel codice a destra.

✗ No	✓ Si
<pre>int check(char m[3][3]) { int i, j; for (i = 0; i < 2; i++) { for (j = 0; j < 2; j++) { if (m[i][j] == 'A') continue; else return 1; } } return 0; }</pre>	<pre>int check(char m[3][3]) { int i, j; for (i = 0; i < 2; i++) { for (j = 0; j < 2; j++) { if (m[i][j] != 'A') return 1; } } return 0; }</pre>

La versione senza `continue` risulta più compatta e ugualmente leggibile.

Occorre sottolineare che, in alcuni casi, eliminare del tutto `break` e `continue` produce codice più complesso e difficile da capire; occorre quindi valutare caso per caso.

👁 Limitare l'uso di *break* e *continue*.

2.5 Array vs Tuple

Un *array* è un tipo di dato in grado di mantenere una sequenza di valori dello stesso tipo; ogni elemento ha lo stesso nome (che coincide con l'identificatore dell'array) ed è univocamente identificato dalla posizione che occupa all'interno della sequenza. Molti linguaggi di programmazione consentono di accedere ad un elemento dell'array in un tempo che non dipende dalla sua posizione. Esempi di array in C:

```
int giorni_mese[12];
float temp_giornaliero[365];
double distanze[10][10];
```

Una *tupla* (detta anche *record*) è un tipo di dato composto da un insieme finito di valori di tipo arbitrario, anche diverso l'uno dall'altro. Ogni elemento di una tupla può essere acceduto, in base al linguaggio di programmazione, usando un nome oppure la posizione che occupa nella tupla. In linguaggio C, le tuple sono definite tramite la parola chiave `struct`, mentre in Java si usano le classi:

✓ Il tipo “persona” in C	✓ Il tipo “persona” in Java
<pre>struct persona { char *nome; char *cognome; int anno_nascita; float peso; };</pre>	<pre>class Persona { String nome; String cognome; int anno_nascita; float peso; };</pre>

Supponiamo di voler rappresentare dei punti materiali nello spazio, ciascuno caratterizzato da posizione, massa e modulo della velocità. La soluzione più appropriata è definire una struttura (in C) o una classe (in Java) contenenti attributi x , y , z , m , v di tipo `float` o `double`. Dato che tutti gli attributi hanno lo stesso tipo, la pigrizia può suggerire di “impacchettarli” in un array di `float` o `double` di lunghezza 5:

✗ No: particelle rappresentate da array	✓ Sì: particelle rappresentate da strutture
<pre>double p1[5], p2[5];</pre>	<pre>struct particle { double x, y, z; /* posizione */ double m; /* massa */ double v; /* velocità */ } p1, p2;</pre>

La rappresentazione basata su array (a sinistra) è inappropriata perché opaca, cioè rende difficile capire il codice che opera sulle particelle. Ad esempio, supponiamo di voler calcolare l'energia cinetica di una particella, definita dall'espressione $\frac{1}{2}mv^2$; in base alla rappresentazione della particella, si ottengono le due implementazioni seguenti:

✗ Cosa calcola?	✓ Calcola l'energia cinetica
<pre>double kinetic_en(double p[]) { return 0.5 * p[3] * (p[4] * p[4]); }</pre>	<pre>double kinetic_en(struct particle p) { return 0.5 * p.m * (p.v * p.v); }</pre>

La funzione a sinistra è poco comprensibile, perché non è evidente cosa rappresentino $p[3]$ e $p[4]$. Nella versione a destra è chiaro che si stanno usando gli attributi m e v della struttura p , i cui nomi suggeriscono si tratti di massa e velocità. Se i nomi degli attributi sono scelti con criterio (vedi 1.4), le tuple rendono il codice più leggibile.

☞ Usare array per raggruppare informazioni logicamente omogenee; usare strutture (C) o classi (Java) per informazioni non omogenee.

2.6 Variabili globali

Una *variabile globale* è visibile (e quindi modificabile) in qualunque punto del codice. L'uso di variabili globali dovrebbe essere limitato a informazioni che vengono effettivamente usate ovunque, tenendo però presente che modificare una variabile globale introduce delle dipendenze tra le funzioni. Consideriamo il seguente frammento di codice C:

```
/* ... */
int f(int x) { ... }
int g(int x) { ... }
/* ... */
int main(void) {
    int x = 10;
    int a = f(x);
    int b = g(x);
    int c = f(x);
    return 0;
}
```

Non sappiamo cosa facciano le funzioni `f()` e `g()`, ma è ragionevole aspettarsi che le variabili `a` e `c` abbiano lo stesso valore, dato che entrambe vengono inizializzate con il valore `f(x)` e il parametro `x` non cambia. Questo non è vero se introduciamo una variabile globale `k` e facciamo dipendere il risultato di `f()` e `g()` da `k`:

```
int k = 0; /* variabile globale */
int f(int x) { return x+k; }
int g(int x) { k++; return x+k; }
int main(void) {
    int x = 10;
    int a = f(x);
    int b = g(x);
    int c = f(x);
    return 0;
}
```

In questo caso si ha che `a` vale 10, `b` vale 11 e `c` vale 11, dato che il valore calcolato dalle funzioni `f()` e `g()` è influenzata da `k`, che viene a sua volta modificata dalle funzioni stesse. In particolare, si vede come il valore dell'espressione `f(x)` cambi da una valutazione all'altra, anche se `x` non viene modificata.

L'uso di variabili globali obbliga chi legge a esaminare il corpo di tutte le funzioni invocate, in quanto non si può assumere che il risultato che restituiscono resti lo stesso a parità di parametri.

☞ Limitare l'uso di variabili globali.

Come sempre, la regola va applicata con ragionevolezza. Esistono molte situazioni in cui l'uso di variabili globali è opportuno, ad esempio:

- programmi relativamente brevi in cui il codice sorgente è distribuito su pochi file (o su un singolo file); in questi casi può essere utile usare variabili globali per ridurre il numero di parametri da passare alle funzioni;
- variabili che vengono effettivamente usate in molte parti del codice.

2.7 Ricorsione

La modifica di variabili globali in funzioni ricorsive è una pessima pratica di programmazione e va evitata. Supponiamo di volere scrivere una funzione ricorsiva `sum(v, n)` che dato un array di interi `v[]` di lunghezza `n`, restituisce la somma dei valori di `v[]` (l'array vuoto ha somma zero). La soluzione ricorsiva si basa sul seguente ragionamento:

- la somma dei valori di un array vuoto vale zero;
- la somma dei valori di un array `v[0..n - 1]` non vuoto è uguale alla somma del sottovettore `v[0..n - 2]` di lunghezza `n - 1`, a cui si somma `v[n - 1]`:

✗ Con variabile globale	✓ Senza variabile globale
<pre>int s; /* Prima di chiamare questa funzione occorre azzerare la variabile globale s */ int sum1(int v[], int n) { if (n == 0) { return 0; } else { s += sum1(v, n-1) + v[n-1]; return s; } }</pre>	<pre>int sum2(int v[], int n) { if (n == 0) return 0; else return sum2(v, n-1) + v[n-1]; }</pre>

Entrambe le soluzioni sfruttano la stessa idea; `sum1()` fa però uso di una variabile globale `s` per accumulare i valori della sommatoria. Questa soluzione è meno leggibile di `sum2()` che non fa uso di variabili globali; inoltre, `sum1()` è estremamente fragile perché ritorna il risultato corretto solo se la variabile `s` viene azzerata prima di invocarla.

L'uso delle variabili statiche del linguaggio C è di fatto equivalente all'uso di variabili globali: anche la soluzione seguente va evitata e per certi versi è perfino peggiore di `sum1()`:

```
int sum3(int v[], int n)
{
    static int s = 0; /* NO!! */

    if (n == 0) {
        return 0;
    } else {
        s += sum3(v, n-1) + v[n-1];
        return s;
    }
}
```

La funzione `sum3()` è peggiore di `sum1()` perché restituisce il valore corretto solo la prima volta che viene invocata, oppure nel caso molto particolare in cui il valore di `s` ritorni zero al termine di una catena di chiamate ricorsive. A differenza di `sum1()`, non è possibile “resettare” una variabile statica dall'esterno di una funzione, quindi soluzioni simili a `sum3()` vanno evitate.

2.8 Asserzioni

Una asserzione è una proprietà che deve essere vera in un certo punto del codice. Molti linguaggi di programmazione mettono a disposizione dei costrutti per verificare la validità delle asserzioni durante l'esecuzione del programma; nel caso in cui una asserzione non sia vera, il programma viene interrotto oppure viene sollevata una eccezione. Le asserzioni sono utili per intercettare gli errori prima che essi si propaghino in punti del codice lontani da dove si è manifestato il problema.

Le asserzioni vengono spesso usate per verificare le *precondizioni* di funzioni o dei metodi. Una *precondizione* è una proprietà che deve essere vera affinché una funzione calcoli il risultato corretto. Ad esempio, consideriamo una funzione C `set_time()` che accetta tre parametri interi rappresentanti l'ora (0–23), i minuti (0–59) e i secondi (0–59) a cui impostare l'orologio di sistema. Possiamo verificare che i parametri siano corretti usando la macro `assert()` definita in `<assert.h>`

```
#include <assert.h>

void set_time(int hh, int mm, int ss)
{
    assert((hh >= 0) && (hh < 24));
    assert((mm >= 0) && (mm < 60));
    assert((ss >= 0) && (ss < 60));
    /* corpo della funzione */
}
```

La macro `assert()` del linguaggio C accetta come unico parametro un valore intero: se il valore è diverso da zero (vero), l'esecuzione del programma prosegue all'istruzione successiva; se il valore è zero (falso) viene stampato a video la riga e il nome del file sorgente in cui si trova la chiamata e il programma termina. Il linguaggio Java mette a disposizione una istruzione `assert` che ha lo stesso comportamento, con la differenza che in Java viene sollevata una eccezione di tipo `java.lang.AssertionError` in caso di asserzione falsa; il programmatore può quindi intercettare l'eccezione e agire di conseguenza senza necessariamente interrompere l'esecuzione. In Java è inoltre necessario eseguire il programma passando il flag `-ea` all'interprete per abilitare il controllo delle asserzioni:

```
java -ea MioProgramma
```

Nell'esempio precedente sarebbe stato possibile accorpare le asserzioni in un'unica espressione:

```
assert((hh >= 0) && (hh < 24) &&
      (mm >= 0) && (mm < 60) &&
      (ss >= 0) && (ss < 60));
```

In questo modo, però, non è possibile capire quale variabile sia responsabile di una eventuale violazione dell'asserzione. Mantenendo le asserzioni separate verrà segnalata la riga di codice responsabile della violazione, il che permetterà di identificare univocamente la variabile coinvolta.

2.9 Invarianti di ciclo

I cicli sono tra le strutture più problematiche di un programma, nel senso che è facile commettere errori e scrivere cicli non corretti, ad esempio sbagliando la condizione di terminazione. La buona notizia è che esistono tecniche matematiche con cui è possibile dimostrare formalmente la correttezza di un ciclo; la cattiva notizia è che tali tecniche diventano rapidamente molto laboriose se applicate a cicli poco più che banali. È tuttavia molto utile capire le basi di queste tecniche, perché nella maggior parte dei casi non serve avere una dimostrazione formale completa.

Tutti i cicli che sono destinati a terminare devono modificare lo stato del programma (ad es., il valore delle variabili) in modo tale che la condizione di terminazione diventi vera. Tuttavia, per dimostrare la correttezza di un ciclo non bisogna concentrarsi su quello che cambia, ma su ciò che resta uguale. L'*invariante* di un ciclo è una proprietà che:

1. è vera prima che il ciclo inizi;
2. rimane vera all'inizio di ogni iterazione del ciclo.

Le due proprietà precedenti assicurano che l'invariante sia vera anche quando il ciclo termina.

In concreto, l'invariante è una espressione logica che deve essere sempre vera prima dell'inizio di un ciclo e prima di ciascuna iterazione. Una trattazione completa dell'argomento esula da questo documento, e si rimanda alla bibliografia. Qui ci limiteremo a fornire un semplice esempio di applicazione delle tecniche di dimostrazione formale di correttezza.

Consideriamo un frammento di codice che, dato un array di interi $a[]$ non vuoto di lunghezza n , determina il minimo valore presente in $a[]$. Il frammento di codice seguente risolve il problema; utilizziamo un ciclo `while` anziché `for` per rendere più chiaro il ragionamento che seguirà.

```
int i = 1, min = a[0];
/* l'invariante I deve valere qui */
while (i < n) {
    /* l'invariante I deve valere qui */
    if (a[i] < min)
        min = a[i];
    i++;
}
```

Abbiamo indicato nei due commenti i punti in cui l'invariante, che chiamiamo I , deve risultare vera. È importante osservare che, in generale, possono esistere più invarianti di uno stesso ciclo; tuttavia, non tutte sono in grado di dimostrare la proprietà che siamo interessati a dimostrare. Nel nostro caso proviamo a considerare quella più semplice, definita come segue:

$$I := \text{"min è il valore minimo presente nel sottovettore } a[0..i - 1]\text{"}$$

Prima dell'inizio del ciclo si ha $i = 1$, $\text{min} = a[0]$, per cui è possibile osservare come l'invariante I sia vera. Si noti che questo richiede che l'array non sia vuoto, cosa che viene garantita dalla specifica del problema.

Per dimostrare che l'invariante è vera all'inizio di ogni iterazione del ciclo, dobbiamo ragionare come segue: supponendo che I valga all'inizio del corpo del ciclo, dimostriamo che I continua a va-

lore alla fine del corpo. Nel nostro caso ci accontentiamo di un ragionamento informale (è comunque possibile procedere in modo più rigoroso) che corrisponde ai commenti nel codice.

```
int i = 1, min = a[0];
/* min è il minimo del sottovettore a[0..i - 1] */
while (i < n) {
    /* min è il minimo del sottovettore a[0..i - 1] */
    if (a[i] < min)
        min = a[i];
    /* min è il minimo del sottovettore a[0..i] */
    i++;
    /* min è il minimo del sottovettore a[0..i - 1] */
}
```

Si noti che prima dell'incremento della variabile i vale la proprietà “ min è il minimo del sottovettore $a[0..i]$ ”, che non è esattamente l'invariante (compare i anziché $i - 1$). Dopo aver incrementato i , la proprietà precedente vale fino all'elemento in posizione precedente il nuovo valore di i , che è proprio l'invariante.

Abbiamo quindi verificato che I gode delle proprietà 1 e 2 descritte sopra, per cui è una invariante per questo ciclo. Di conseguenza, sarà valida anche al termine dello stesso, assieme alla proprietà ($i \geq n$) che è la negazione della condizione del `while`. Possiamo perciò concludere che, alla fine del ciclo si ha:

min è il minimo del sottovettore $a[0..i - 1]$ e $i \geq n$

Da questa non è però possibile concludere che min sia il minimo dell'intero array, perché per affermare ciò bisognerebbe avere la garanzia che $i = n$. Il problema deriva dalla scelta dell'invariante, che è troppo debole per quello che vogliamo dimostrare. È però possibile ripetere il ragionamento con una nuova invariante I_2 definita come:

$I_2 :=$ “ min è il valore minimo presente nel sottovettore $a[0..i - 1]$ e $i \leq n$ ”

Possiamo verificare che I_2 risulti valida prima dell'inizio del ciclo e all'inizio del corpo dello stesso:

```
int i = 1, min = a[0];
/* min è il minimo del sottovettore a[0..i - 1] e i <= n */
while (i < n) {
    /* min è il minimo del sottovettore a[0..i - 1] e i < n */
    if (a[i] < min)
        min = a[i];
    /* min è il minimo del sottovettore a[0..i] e i < n */
    i++;
    /* min è il minimo del sottovettore a[0..i - 1] e i <= n */
}
```

La dimostrazione precedente deve tenere conto di un paio di punti importanti. La specifica del problema garantisce che l'array sia sempre non vuoto ($n \geq 1$); di conseguenza, l'invariante I_2 risulta vera prima del ciclo. Si noti che all'inizio del corpo del ciclo vale una proprietà in apparenza diversa dall'invariante, perché abbiamo scritto $i < n$ anziché $i \leq n$. Il motivo è che la condizione del ciclo è

($i < n$), per cui sappiamo che entrando nel ciclo questa proprietà è sicuramente vera. Tuttavia, sapendo che vale:

min è il minimo del sottovettore $a[0..i - 1]$ e $i < n$

allora sarà sicuramente vera l'invariante I_2 , dato che, se $i < n$, sarà anche vero che $i \leq n$. Di conseguenza, all'inizio del ciclo vale una proprietà più stringente di I_2 , nel senso che il fatto che quella proprietà sia vera garantisce che anche I_2 lo sia.

Possiamo concludere che I_2 soddisfa le proprietà di una invariante, per cui alla fine del ciclo si avrà:

min è il minimo del sottovettore $a[0..i - 1]$ e $i \leq n$ e $i \geq n$

L'espressione $i \leq n$ fa parte di I_2 , mentre $i \geq n$ deriva dal fatto che al termine del ciclo vale la negazione della condizione del `while`. Poiché sono vere entrambe, possiamo dedurre che $i = n$, da cui otteniamo

min è il minimo del sottovettore $a[0..i - 1]$ e $i = n$

che è la proprietà che vogliamo dimostrare.

Sebbene il ragionamento precedente possa essere applicato a cicli più complessi, le dimostrazioni formali di correttezza tendono a diventare rapidamente ingestibili. Tuttavia, nella pratica è sufficiente avere consapevolezza di quale sia la proprietà che vogliamo resti vera durante l'esecuzione di un ciclo. In questo modo, durante la stesura del codice possiamo prestare attenzione a garantire che l'invariante resti vera e il ciclo sia corretto.

 Individuare l'invariante di ciascun ciclo.

2.10 Programmazione difensiva

Il termine “programmazione difensiva” fa riferimento alle tecniche per sviluppare programmi che reagiscono in modo controllato a situazioni impreviste. “Reagire in modo controllato” non significa necessariamente fornire il risultato corretto, bensì evitare che le anomalie si propaghino ad altre parti del programma rendendolo instabile. A differenza delle asserzioni, la programmazione difensiva deve assicurare che il programma non si interrompa in presenza di condizioni impreviste. Ciò è desiderabile, ad esempio, in sistemi critici (es., sistemi di controllo di un impianto industriale) il cui funzionamento ininterrotto è essenziale.

Consideriamo i due frammenti di codice seguenti:

 Potrebbe non terminare	 Termina sempre
<pre>int i, x = ...; for (i = x; i != n-1; i = i+2) { blah(i); }</pre>	<pre>int i, x = ...; for (i = x; i < n; i = i+2) { blah(i); }</pre>

La variabile i viene incrementata di due ad ogni iterazione. Se x ed n hanno la stessa parità (entrambi pari o entrambi dispari) il ciclo di sinistra non termina, come si può facilmente verificare con $x = 2, n = 4$. Il ciclo di sinistra non termina anche se x è positivo e n negativo, a prescindere dalla loro parità.

L'applicazione del principio di programmazione difensiva porta a modificare la condizione `i != n - 1` in `i < n`. Facendo in questo modo il ciclo termina qualunque sia il valore iniziale x ; naturalmente non è detto che il programma fornisca il risultato corretto, ma si evita il rischio di un ciclo che potrebbe essere infinito oppure no in base a come il programma e/o l'hardware gestiscono l'overflow della variabile indice i .

Come ulteriore esempio si considerino le seguenti implementazioni di una funzione ricorsiva `fib(n)` che calcola l' n -esimo numero della successione di Fibonacci (`fib(0)` è il primo numero della successione).

✗ Comportamento indefinito se $n < 0$	✓ Termina sempre
<pre>int fib(int n) { if ((n == 0) (n == 1)) { return 1; } else { return fib(n-1) + fib(n-2); } }</pre>	<pre>int fib(int n) { if (n <= 1) { return 1; } else { return fib(n-1) + fib(n-2); } }</pre>

La versione di sinistra ha un comportamento indefinito se $n < 0$, perché le chiamate ricorsive si allontanano sempre di più dal caso base anziché avvicinarsi. La terminazione della funzione dipende da come il programma e/o l'hardware gestiscono l'*underflow* della variabile n , cioè cosa succede quando le espressioni $n - 1$ oppure $n - 2$ hanno un valore negativo troppo piccolo per essere rappresentato dal tipo `int`. La versione a destra rimpiazza la condizione `((n == 0) || (n == 1))` con `n <= 1`: in questo modo la funzione termina sempre, restituendo 1 quando $n < 0$. Nel caso in cui il caso $n < 0$ rappresenti un errore che deve essere segnalato, si può aggiungere l'asserzione `assert(n >= 0)` all'inizio della funzione `fib()`. In questo modo la condizione anomala causerebbe l'interruzione del programma.

🔒 Programmare in modo difensivo.

2.11 Evitare soluzioni “astute”

Talvolta capita di farsi prendere la mano esibendo il proprio virtuosismo per proporre “soluzioni astute” a problemi semplici. Col termine “astute” si intende soluzioni contorte e difficili da comprendere a problemi banali che ammettono una soluzione ovvia. Quasi sempre, le soluzioni astute non si rivelano né eleganti né compatte, e spesso nemmeno del tutto corrette. **Proporre soluzioni contorte a problemi semplici non è motivo di vanto.** Un bravo programmatore o programmatrice sviluppa soluzioni comprensibili, facilmente modificabili, ed efficienti. Un programmatore scarso tende a nascondere la propria mediocrità in esibizioni di virtuosismo fuori luogo. Vediamo alcuni esempi.

Consideriamo tre variabili a, b, c di tipo `char`. Supponiamo che le variabili possano assumere esclusivamente i valori 'X' oppure 'Y'. Si chiede di scrivere un frammento di codice per verificare se tutte e tre le variabili hanno valore 'X'.

La soluzione ovvia è l'espressione `(a == 'X') && (b == 'X') && (c == 'X')`. Tuttavia, è anche possibile immaginare una soluzione “astuta” in cui si effettua un solo confronto anziché tre.

✗ Soluzione "astuta"	✓ Soluzione leggibile
<pre>if (a + b + c == 3 * 'X') { printf("tutti uguali a X\n"); } else { printf("non tutti uguali a X\n"); }</pre>	<pre>if ((a == 'X') && (b == 'X') && (c == 'X')) { printf("tutti uguali a X\n"); } else { printf("non tutti uguali a X\n"); }</pre>

La soluzione “astuta” sfrutta il fatto che, *nell'ipotesi in cui i valori possano essere solo 'X' oppure 'Y'*, le variabili hanno lo stesso valore se e solo se la somma dei codici ASCII dei caratteri in esse contenuti è il triplo del codice ASCII del carattere 'X'. Sebbene ciò sia corretto in questo caso, si può osservare come la soluzione “astuta” non sia né significativamente più compatta né più leggibile di quella normale. A differenza di quella normale, però, non è immediatamente comprensibile, e costringe chi legge il codice a lunghi ragionamenti per intuire cosa si sta facendo. Inoltre, la soluzione “astuta” non è generalizzabile al caso in cui le variabili possano contenere caratteri arbitrari. Ad esempio, se $a = 'w', b = 'x', c = 'y'$, dato che i codici ASCII dei caratteri sono consecutivi, l'espressione `a + b + c == 3 * 'X'` risulta vera ma le tre variabili non contengono tutte 'X'. La soluzione “astuta” è quindi opaca (difficile da comprendere) e fragile (non generalizzabile al di fuori di questo caso particolare).

Come altro esempio consideriamo il problema di scambiare tra loro il contenuto di due variabili x e y di tipo `int`. La versione “astuta” non fa uso di alcuna variabile di appoggio:

✗ Soluzione "astuta"	✓ Soluzione leggibile
<pre>x = x - y; y = x + y; x = y - x;</pre>	<pre>const int tmp = x; x = y; y = tmp;</pre>

Se non si presta attenzione si potrebbe riuscire a dimostrare formalmente che la soluzione “astuta” sia corretta, nel senso che scambi sempre i valori di x e y senza fare uso di variabili temporanee. Purtroppo le dimostrazioni semplici trascurano un aspetto fondamentale: il tipo `int` del linguaggio C consente di rappresentare un intervallo *limitato* di valori (di solito quelli nell'intervallo $[-2^{31}, 2^{31} - 1]$ se il tipo `int` ha ampiezza 32 bit e la rappresentazione è in *complemento a due*). Al contrario, quando si fanno dimostrazioni matematiche si assume sempre che i simboli (le variabili) possano assumere valori arbitrariamente grandi o piccoli. Nella linguaggio C, se due variabili intere x e y contengono una un valore molto grande e l'altra un valore molto piccolo, l'espressione `x - y` po-

trebbe causare *overflow*⁴ e il risultato della computazione sarebbe indefinito. La soluzione diretta, basata sull'uso di una variabile temporanea non soffre di tale problema.

Come altro esempio consideriamo le due implementazioni equivalenti della stessa funzione `f()`. Che cosa calcola?

✖ Soluzione "astuta"	✔ Soluzione leggibile
<pre>int f(int x) { return (x > 0) - (x < 0); }</pre>	<pre>int f(int x) { if (x > 0) return 1; else if (x < 0) return -1; else return 0; }</pre>

La versione leggibile rende evidente che `f(x)` è il segno di x , che vale 1 se x è positivo, -1 se x è negativo, e 0 se x vale zero. La versione “astuta” calcola lo stesso risultato⁵, ma richiede uno sforzo notevole per essere compresa.

Vediamo un ultimo esempio descritto da Kernighan e Plauger nel libro menzionato in bibliografia. Consideriamo il seguente frammento di codice:

```
#define N 10
int mat[N][N];
int i, j;

for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        mat[i][j] = ((i+1)/(j+1))*((j+1)/(i+1));
    }
}
```

Il codice inizializza la matrice `mat[][]`, ma non è chiaro con quali valori. Per capirlo osserviamo che:

- Poiché i, j sono interi, allora i rapporti `(i+1)/(j+1)` e `(j+1)/(i+1)` vengono valutati secondo le regole del linguaggio C in base alle quali il rapporto tra due interi è un intero ottenuto troncando il risultato;
- Se $i = j$, allora l'espressione `((i+1)/(j+1))*((j+1)/(i+1))` vale 1 perché i numeratori e denominatori sono uguali;
- Se $i \neq j$, allora dobbiamo distinguere due sotto casi:

4 A lezione utilizzo questo esempio per mostrare quanto sia difficile fare dimostrazioni formali di correttezza. Infatti, assumendo che x e y vengano rappresentate con precisione infinita, è vero che il codice “astuto” produce sempre il risultato corretto. Purtroppo, nella maggior parte dei linguaggi di programmazione tale assunzione non è vera. Se la si incorpora nella dimostrazione di correttezza -- che diventa così molto più complessa -- si scopre che esistono casi in cui il frammento di codice non produce il risultato atteso.

5 La versione “astuta” potrebbe risultare più efficiente su certe architetture hardware, perché evita le istruzioni di salto condizionato che nei moderni processori comportano delle inefficienze. Tuttavia, nella maggior parte delle applicazioni il miglioramento è probabilmente insignificante e non compensato dalla perdita di leggibilità.

- $i < j$: in questo caso $(i+1)/(j+1)$ vale zero, perché il numeratore è minore del denominatore. Di conseguenza, a $mat[i][j]$ viene assegnato il valore zero
- $i > j$: in questo caso $(j+1)/(i+1)$ vale zero, perché il numeratore è minore del denominatore. Anche in questo caso a $mat[i][j]$ viene assegnato il valore zero

In sostanza, il frammento di codice precedente inizializza la matrice in modo che contenga tutti zeri tranne che nella diagonale principale che contiene valori uno. In altre parole, $mat[][]$ viene inizializzata come la matrice identità.

Esistono moltissimi modi, molto più chiari e altrettanto efficienti, per ottenere lo stesso risultato. Ad esempio:

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        if (i == j)
            mat[i][j] = 1;
        else
            mat[i][j] = 0;
    }
}
```

oppure, in modo più compatto ma meno leggibile:

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        mat[i][j] = (i == j ? 1 : 0);
    }
}
```

oppure, in modo ancora più compatto ma ancor meno leggibile:

```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        mat[i][j] = (i == j);
    }
}
```

Tutte le versioni precedenti sono preferibili rispetto a quella "astuta".

 Evitare soluzioni "astute".

2.12 Stack vs Heap

In alcuni linguaggi di programmazione come C/C++, le variabili locali vengono allocate nello *stack*. In alcuni sistemi operativi esiste un limite alla dimensione massima dello stack, che potrebbe essere modificabile dall'utente o dall'amministratore di sistema. Ad esempio, il comando `ulimit -a` in ambiente Linux visualizza i limiti di sistema, e sulla mia macchina il comando produce l'output seguente:

```
core file size      (blocks, -c) 0
data seg size       (kbytes, -d) unlimited
scheduling priority (-e) 0
file size            (blocks, -f) unlimited
```

```

pending signals          (-i) 127624
max locked memory        (kbytes, -l) 65536
max memory size          (kbytes, -m) unlimited
open files               (-n) 1024
pipe size                (512 bytes, -p) 8
POSIX message queues     (bytes, -q) 819200
real-time priority       (-r) 0
stack size               (kbytes, -s) 8192
cpu time                 (seconds, -t) unlimited
max user processes       (-u) 127624
virtual memory           (kbytes, -v) unlimited
file locks               (-x) unlimited

```

da cui si vede che la dimensione massima dello stack è 8192 KByte (8 Mbyte).

La dimensione massima dello stack non dovrebbe rappresentare un problema nella maggior parte delle situazioni. Tuttavia, può causare problemi quando si definisce un array o una matrice di grandi dimensioni. Ad esempio, il frammento di codice seguente:

```

#define N ...
int pippo(void)
{
    float tmp[N]; /* Può fallire se N è troppo grande */
    /* ... */
}

```

potrebbe causare una interruzione anomala se il valore di *N* è troppo elevato. dato che lo spazio richiesto dall'array `tmp[]` *potrebbe* eccedere lo spazio disponibile nello stack. Questo tipo di errore è insidioso, sia perché potrebbe manifestarsi o meno in base a come è configurato il sistema operativo, ma anche perché il problema potrebbe manifestarsi non nel punto del programma in cui si dichiara l'array, ma nel punto in cui si tenta di accedere al suo contenuto.

Questo tipo di problema si potrebbe manifestare anche usando i *variable length array* del C99. Nell'esempio seguente si alloca un array di interi la cui dimensione *n* viene letta da standard input; la dichiarazione `int tmp[n]` potrebbe fallire se *n* è troppo grande. Se non si ha la garanzia che la lunghezza dell'array sia sempre limitata, è preferibile allocare `tmp[]` nello heap utilizzando la funzione `malloc()`, che fallisce solo quando si esaurisce la memoria RAM di sistema.

✗ Può fallire se <i>n</i> è "grande"	✓ Ok
<pre> int main(void) { int n; scanf("%d", &n); int tmp[n]; /* qui si usa tmp[] */ return 0; } </pre>	<pre> int main(void) { int n; scanf("%d", &n); int *tmp = (int*)malloc(n * sizeof(int)); /* qui si usa tmp[] */ free(tmp); return 0; } </pre>

🔗 Allocare array o matrici di grandi dimensioni sullo heap, non sullo stack.

2.13 Macro

Il linguaggio C/C++ include un preprocessore che applica delle trasformazioni al codice sorgente prima della compilazione vera e propria. Il preprocessore offre la possibilità di definire delle macro, ossia dei nomi a cui corrispondono delle sequenze di istruzioni. A differenza delle funzioni, le macro vengono espanse nel punto in cui sono invocate, nel senso che il preprocessore sostituisce a ciascuna occorrenza di una macro la sequenza di simboli ad essa associati.

Spesso si tende ad utilizzare macro al posto di funzioni. Ciò non è una buona idea, perché l'invocazione di macro è soggetta a regole diverse rispetto all'invocazione di funzioni, e questo può portare ad errori insidiosi. Un esempio classico è la macro seguente, che dovrebbe restituire il quadrato dell'argomento:

```
#define POW(x) (x*x)
```

Invece, l'espressione `POW(8 + 2)` non ha valore 100 (il quadrato di $8 + 2 = 10$), ma 26, dato che la macro viene espansa in:

```
8 + 2 * 8 + 2
```

Un altro esempio è la macro `MAX(a, b)` seguente, che dovrebbe restituire il valore massimo tra gli argomenti *a*, *b*:

```
#define MAX(a, b) (a) > (b) ? (a) : (b)
```

Questa definizione non è corretta, perché l'espressione `MAX(4, 2) + 1` restituisce 4, non 5 come ci si aspetta; il motivo è che l'espressione viene espansa in:

```
(4) > (2) ? (4) : (2) + 1
```

ma a causa della priorità degli operatori, è come se fosse scritta:

```
4 > 2 ? 4 : (2 + 1)
```

Per questo motivo si trovano spesso delle parentesi in apparenza superflue nelle dichiarazioni delle macro:

```
#define POW(x) ((x)*(x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

Quanto sopra non è tuttavia sufficiente, dato che le macro si prestano ad errori molto più subdoli. Ad esempio, consideriamo nuovamente la macro `MAX(a, b)` definita correttamente come:

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

Consideriamo ora il seguente frammento di codice:

```
int x = 10;
int y = 32;
int z = MAX(x++, y++);
```

Se `MAX()` fosse una funzione, ci aspetteremmo che al termine dell'esecuzione delle istruzioni si abbia $z = 32$, $x = 11$ e $y = 33$. In realtà, nelle macro si applica la *sostituzione lessicale dei parametri*, cioè le espressioni `x++` e `y++` vengono sostituite al posto dei parametri a e b della macro. Di conseguenza, il codice precedente viene espanso in:

```
int x = 10;
int y = 32;
int z = ((x++) > (y++) ? (x++) : (y++));
```

e si vede che la variabile che contiene il massimo tra x e y viene incrementata *due* volte.

L'uso delle macro era giustificato in passato da ragioni di efficienza: le macro evitano l'overhead legato alle chiamate di funzione (gestione dello stack, passaggio dei parametri, eccetera). Oggi queste giustificazioni non sono più valide, perché i compilatori moderni sono in grado di generare codice altamente ottimizzato. Per evitare sorprese è quindi opportuno definire `MAX()` come una normale funzione:

```
int MAX(int a, int b)
{
    return (a > b ? a : b);
}
```

 Dove possibile, usare funzioni anziché macro.

3 Esempi

*In theory, theory equals practice
In practice, they differ*

In questo capitolo prendiamo in esame alcuni frammenti di codice reale, trovato in rete o su libri e riviste, per dimostrare come le tecniche descritte in precedenza possano essere applicate.

3.1 Calibrazione di un anemometro

Il frammento di codice seguente compare in un programma eseguito da un microcontrollore per calcolare la velocità del vento partendo dal numero di impulsi di un anemometro; si può assumere che la variabile *frequency* sia di tipo `int` e *correction* sia di tipo `float` se la frequenza è minore di 0 o maggiore di 1600 assumiamo che la variabile *correction* debba valere zero. I dettagli del funzionamento dell'anemometro non sono importanti per i nostri scopi.

```
void adjustments( void )
{
    if ((frequency >=1) && (frequency <=100)) {correction = 0.20;}
    if ((frequency >100) && (frequency <=200)) {correction = 0.10;}
    if ((frequency >200) && (frequency <=300)) {correction = -0.25;}
    if ((frequency >300) && (frequency <=400)) {correction = -0.60;}
    if ((frequency >400) && (frequency <=500)) {correction = -1;}
    if ((frequency >500) && (frequency <=600)) {correction = -1.4;}
    if ((frequency >600) && (frequency <=700)) {correction = -1.75;}
    if ((frequency >700) && (frequency <=800)) {correction = -2;}
    if ((frequency >800) && (frequency <=900)) {correction = -2.20;}
    if ((frequency >900) && (frequency <=1000)) {correction = -2.35;}
    if ((frequency >1000) && (frequency <=1100)) {correction = -2.40;}
    if ((frequency >1100) && (frequency <=1200)) {correction = -2.50;}
    if ((frequency >1200) && (frequency <=1300)) {correction = -2.50;}
    if ((frequency >1300) && (frequency <=1400)) {correction = -2.50;}
    if ((frequency >1400) && (frequency <=1500)) {correction = -2.45;}
    if ((frequency >1500) && (frequency <=1600)) {correction = -2.40;}
}
```

Questo è un esempio “da manuale” in cui si dovrebbe spostare la logica dal codice ai dati, come illustrato nella sezione 2.1, Osserviamo che gli intervalli dei valori della variabile *frequency* che corrispondono alle varie correzioni sono tutti del tipo $(100i, 100(i + 1)]$. È quindi possibile definire un array *corrections*[] di lunghezza 16, tale che *corrections*[*i*] sia la correzione da applicare alle frequenze comprese tra $100i$ (escluso) e $100(i + 1)$ (compreso). Poiché si suppone di conoscere la

frequenza, l'indice dell'elemento dell'array corrispondente alla correzione può essere calcolato come `frequency / 100`. Otteniamo quindi il codice seguente:

```
void adjustments( void )
{
    static const float corrections[] = {0.20, 0.10, -0.25, -0.60, ... -2.45, -
2.40};
    if (frequency < 0 || frequency > 1600) {
        correction = 0.0;
    } else {
        correction = corrections[frequency / 100];
    }
}
```

3.2 I led lampeggianti

Il codice che segue fa parte di un programma per un microcontrollore; il programma attiva un led di colore diverso in base al valore della variabile *WarnLevel*. `digitalWrite()` è una funzione predefinita nella libreria del microcontrollore, come pure i simboli `LED_green`, `LED_yellow`, `LED_red`, `LOW` e `Blink`. Possiamo assumere che questi simboli siano associati a valori di tipo `int`. Non possiamo assumere nulla circa lo stato iniziale dei led, per cui quelli che vanno tenuti spenti devono essere impostati esplicitamente allo stato `LOW`.

```
if (WarnLevel < 1000) {
    digitalWrite(LED_green, LOW);
    digitalWrite(LED_yellow, LOW);
    digitalWrite(LED_red, LOW);
}
if ((WarnLevel >= 1000) && (WarnLevel < 1500)){
    digitalWrite(LED_green, Blink);
    digitalWrite(LED_yellow, LOW);
    digitalWrite(LED_red, LOW);
}
if ((WarnLevel >= 1500) && (WarnLevel < 2500)) {
    digitalWrite(LED_green, LOW);
    digitalWrite(LED_yellow, Blink);
    digitalWrite(LED_red, LOW);
}
if (WarnLevel >= 2500){
    digitalWrite(LED_green, LOW);
    digitalWrite(LED_yellow, LOW);
    digitalWrite(LED_red, Blink);
}
```

Analizzando il codice vediamo che il comportamento del programma è il seguente:

- se *WarnLevel* è minore di 1000, non accende nessun led;
- se *WarnLevel* è tra 1000 (compreso) e 1500 (escluso), fa lampeggiare il led verde;
- se *WarnLevel* è tra 1500 (compreso) e 2500 (escluso), fa lampeggiare il led giallo;

- se *WarnLevel* è maggiore o uguale a 2500, fa lampeggiare il led rosso.

Le condizioni sono scritte in cascata in un modo che non rende evidente che siano mutuamente esclusive. Possiamo rendere chiaro che solo una di esse può risultare vera utilizzando clausole **else**:

```
if (WarnLevel < 1000) {
    digitalWrite(LED_green, LOW);
    digitalWrite(LED_yellow, LOW);
    digitalWrite(LED_red, LOW);
} else if (WarnLevel < 1500) {
    digitalWrite(LED_green, Blink);
    digitalWrite(LED_yellow, LOW);
    digitalWrite(LED_red, LOW);
} else if (WarnLevel < 2500) {
    digitalWrite(LED_green, LOW);
    digitalWrite(LED_yellow, Blink);
    digitalWrite(LED_red, LOW);
} else {
    digitalWrite(LED_green, LOW);
    digitalWrite(LED_yellow, LOW);
    digitalWrite(LED_red, Blink);
}
```

(dato che le condizioni sono poche, non vale la pena adottare la tecnica descritta nella Sezione 2.1 per sfruttare la ricerca binaria). Osserviamo ora che le chiamate `digitalWrite()` vengono ripetute in modo molto simile in tutti i blocchi. Questo suggerisce una rifattorizzazione del codice, che può essere realizzata in modi diversi. Una possibilità è usare un array *leds[]* di lunghezza 3 in cui memorizzare lo stato dei led (inizialmente LOW):

```
int leds[3] = {LOW, LOW, LOW};

if (WarnLevel < 1000)
    /* vuoto */
else if (WarnLevel < 1500)
    leds[0] = Blink;
else if (WarnLevel < 2500)
    leds[1] = Blink;
else
    leds[2] = Blink;

digitalWrite(LED_green, leds[0]);
digitalWrite(LED_yellow, leds[1]);
digitalWrite(LED_red, leds[2]);
```

Possiamo rendere l'intento del codice ancora più evidente, assegnando un nome simbolico agli indici dell'array *leds[]* perché non è immediatamente chiaro che, ad esempio, *leds[0]* sia lo stato del led verde. Otteniamo quindi:

```
enum { GREEN, YELLOW, RED };
int leds[3] = {LOW, LOW, LOW};

if (WarnLevel < 1000)
    /* niente */
else if (WarnLevel < 1500)
    leds[GREEN] = Blink;
```

```

else if (WarnLevel < 2500)
    leds[YELLOW] = Blink;
else
    leds[RED] = Blink;

digitalWrite(LED_green, leds[GREEN]);
digitalWrite(LED_yellow, leds[YELLOW]);
digitalWrite(LED_red, leds[RED]);

```

Un'altra possibilità è definire una funzione ausiliaria `blink()` a cui delegare l'impostazione dello stato dei led tramite una *bitmask* in cui il valore dei tre bit meno significativi rappresenta lo stato desiderato dei led (1 = led lampeggiante, 0 = led non lampeggiante). Per costruire la bitmask è utile definire tre simboli `BLINK_GREEN`, `BLINK_YELLOW` e `BLINK_RED` i cui valori sono potenze di due:

```

enum { BLINK_GREEN = 1, BLINK_YELLOW = 2, BLINK_RED = 4 };

/* Fai lampeggiare i led corrispondenti ai bit posti a 1 in `flags`; questo parametro viene interpretato come
una bitmask partendo dalle costanti BLINK_GREEN, BLINK_YELLOW e BLINK_RED */
void blink(int flags)
{
    digitalWrite(LED_green, (flags & BLINK_GREEN) ? Blink : LOW);
    digitalWrite(LED_yellow, (flags & BLINK_YELLOW) ? Blink : LOW);
    digitalWrite(LED_red, (flags & BLINK_RED) ? Blink : LOW);
}

if (WarnLevel < 1000)
    blink(0);
else if (WarnLevel < 1500)
    blink(BLINK_GREEN);
else if (WarnLevel < 2500)
    blink(BLINK_YELLOW);
else
    blink(BLINK_RED);

```

Questa soluzione è più generale perché consente di far lampeggiare più di un led nel caso in cui l'applicazione lo richieda. Ad esempio, `blink(BLINK_GREEN | BLINK_YELLOW)` fa lampeggiare i led verde e giallo, mentre `blink(BLINK_GREEN | BLINK_YELLOW | BLINK_RED)` li fa lampeggiare tutti e tre.

3.3 Lo switch-case infinito

La funzione seguente è presente in un programma eseguito su un microcontrollore, e ha lo scopo di pilotare una matrice di led; i simboli `A0`, `A1`, `A2` sono costanti definite altrove.

Non serve elencare i problemi del codice: basta guardarlo per capire che deve essere possibile fare meglio di così (molti programmi per microcontrollore hanno qualità discutibile, forse perché sono scritti da esperti di elettronica che non necessariamente sanno programmare).

```

void MatrixON(int X, int Y)
{
    int xx;
    int yy;

    switch (Y) {
        case 1:
            yy=2;
            break;
        case 2:
            yy=3;
            break;
        case 3:
            yy=4;
            break;
        case 4:
            yy=5;
            break;
        case 5:
            yy=6;
            break;
        case 6:
            yy=7;
            break;
        case 7:
            yy=8;
            break;
        case 8:
            yy=9;
            break;
        case 9:
            yy=10;
            break;
        case 10:
            yy=11;
            break;
        case 11:
            yy=12;
            break;
        case 12:
            yy=13;
            break;
        case 13:
            yy=A0;
            break;
        case 14:
            yy=A1;
            break;
        case 15:
            yy=A2;
            break;
        default:
            yy=0;
            break;
    }

    switch (X) {
        case 1:
            xx=A2;
            break;
        case 2:
            xx=A2;
            break;
        case 3:
            xx=A1;
            break;
        case 4:
            xx=A1;
            break;
        case 5:
            xx=A0;
            break;
        case 6:
            xx=A0;
            break;
        case 7:
            xx=13;
            break;
        case 8:
            xx=13;
            break;
        case 9:
            xx=12;
            break;
        case 10:
            xx=12;
            break;
        case 11:
            xx=11;
            break;
        case 12:
            xx=11;
            break;
        case 13:
            xx=10;
            break;
        case 14:
            xx=10;
            break;
        case 15:
            xx=9;
            break;
        case 16:
            xx=9;
            break;
        case 17:
            xx=8;
            break;
        case 18:
            xx=8;
            break;
        case 19:
            xx=7;
            break;
        case 20:
            xx=7;
            break;
        case 21:
            xx=6;
            break;
        case 22:
            xx=6;
            break;
        case 23:
            xx=5;
            break;
        case 24:
            xx=5;
            break;
        case 25:
            xx=4;
            break;
        case 26:
            xx=4;
            break;
        case 27:
            xx=3;
            break;
        case 28:
            xx=3;
            break;
        default:
            xx=0;
            break;
    }
}
/* resto del
codice */

```

Analizzando il codice, osserviamo che il primo `switch-case` assegna a `yy` un valore intero che dipende da `Y`; a sua volta, la variabile `Y` può assumere valori compresi tra 1 e 15 (estremi inclusi); se il valore di `Y` è fuori da questo intervallo, la variabile `yy` viene posta a zero. In modo analogo, il secondo `switch-else` assegna a `xx` un valore intero che dipende dal `X`; la variabile `X` può assumere valori compresi tra 1 e 28 (estremi inclusi). Come nel caso precedente, se `X` ha un valore fuori da tale intervallo, `xx` viene posta a zero.

Si può notare come ci sia una dipendenza abbastanza semplice tra `Y` e `yy`. Ad esempio, se `Y` appartiene all'insieme $\{1, \dots, 12\}$, allora $yy = Y + 1$. Tuttavia, la dipendenza di `xx` da `X` è molto meno semplice, per cui risulta conveniente ragionare come se il valore di `yy` (e `xx`) non possa essere espresso in modo “semplice” come funzione di `Y` (e `X`).

Il codice precedente è un ottimo candidato per trasferire la logica dal programma ai dati (Sezione 2.1). A tale scopo, dichiariamo un array `yyval[15]` tale che `yyval[k]` sia il valore da assegnare a `yy` nel caso in cui $Y == k + 1$; in modo analogo definiamo un array `xxval[28]`. Il codice diventa:

```

void MatrixON(int X, int Y)
{
    int xx = 0; /* valore di default */
    int yy = 0; /* valore di default */
    const int yyval[15] = { 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
                           12, 13, A0, A1, A2};
    const int xxval[28] = {A2, A2, A1, A1, A0, A0, 13, 13, 12, 12,
                           11, 11, 10, 10, 9, 9, 8, 8, 7, 7,
                           6, 6, 5, 5, 4, 4, 3, 3};

    if (Y >= 1 && Y <= 15)
        yy = yyval[Y-1];
    if (X >= 1 && X <= 28)
        xx = xxval[X-1];
    /* resto del codice */
}

```

3.4 Controlla la data

Riportiamo un frammento di codice C che è stato convertito da un programma in linguaggio BASIC trovato su un vecchio libro. Il codice chiede all'utente di inserire un anno compreso tra 1900 e 2300 inclusi, stampando un messaggio di errore e ripetendo la richiesta in caso di valore non valido.

```

do {
    do {
        printf("Inserire anno");
        scanf("%d", &y);
        if (y<1900 || y>2300)
            printf("Errore. Reinserire\n");    /* (1) */
    } while (y<1900 || y>2300);
    if (y<1900 || y>2300)    /* (2) */
        printf("Errore. Reinserire\n");
} while (y < 0);

```

Osserviamo che il test (2) è inutile, perché l'unico modo di uscire dal ciclo che lo precede è che la condizione (1) sia falsa. Questo significa che subito dopo il ciclo `while` più interno vale la proprietà `(y >= 1900 && y <= 2300)`. Di conseguenza la condizione (2) sarà sempre falsa e può essere omessa:

```

do {
    do {
        printf("Inserire anno");
        scanf("%d", &y);
        if (y<1900 || y>2300)
            printf("Errore. Reinserire\n");
    } while (y<1900 || y>2300);
} while (y < 0);

```

Notiamo ora che il ciclo `do-while` più esterno può essere eliminato, perché al termine di quello interno si ha la garanzia che l'espressione `(y >= 1900 && y <= 2300)` sia vera, per cui non si potrà mai avere `y<0`. Il codice può quindi essere ulteriormente semplificato in:

```

do {
    printf("Inserire anno");

```

```
scanf("%d", &y);
if (y<1900 || y>2300)
    printf("Errore. Reinserire\n");
} while (y<1900 || y>2300);
```

3.5 sc

Illustriamo ora un esempio complesso in cui è possibile eliminare `break` e `continue` ma a costo di rendere il codice più complesso da capire; inoltre, l'eliminazione dei due costrutti richiede attenzione e può introdurre errori se non fatto correttamente. Come già detto in altre occasioni, le regole descritte in questa dispensa vanno applicate con ragionevolezza.

Il frammento di codice che segue compare nei sorgenti di `sc`, un foglio di calcolo con interfaccia a caratteri usato in ambiente Unix.

```
for (tx = stcol, i = x; ; tx++) {
    if (col_hidden[tx])
        continue;
    if ((i -= fwidth[tx]) < 0)
        break;
}
```

Il codice mostra due aspetti problematici: la condizione del ciclo `for` è vuota (si veda 2.3), e sono presenti istruzioni `continue` e `break` (si veda 2.4). In particolare, la presenza di `continue` e `break` complica l'analisi del ciclo perché non è immediato determinare l'invariante. Il ciclo è insidioso perché bisogna ricordare che `break` causa l'uscita dal ciclo senza eseguire l'incremento `tx++`; quando si ristrutturava un ciclo, occorre assicurarsi che al termine i valori delle variabili siano identici a quelli del ciclo originale.

Eliminare `continue` è relativamente facile, dopo aver osservato che il ciclo può essere riscritto come:

```
for (tx = stcol, i = x; ; tx++) {
    if (col_hidden[tx]) {
        /* vuoto */
    } else {
        if ((i -= fwidth[tx]) < 0)
            break;
    }
}
```

da cui è possibile eliminare il ramo “vero” della condizione (sezione 2.2) ottenendo:

```
for (tx = stcol, i = x; ; tx++) {
    if (!col_hidden[tx] && ((i -= fwidth[tx]) < 0)) /* condizione C */
        break;
}
```

Rimuovere l'istruzione `break` è più laborioso; in casi come questo, se non si è sicuri che le trasformazioni da applicare mantengano invariato il comportamento del codice, può essere consigliabile lasciare tutto come sta. Chiamiamo C l'espressione:

```
(!col_hidden[tx] && ((i -= fwidth[tx]) < 0))
```

Osserviamo che il ciclo termina quando C è vera, per cui il ciclo continua fintanto che C rimane falsa. Possiamo quindi scrivere:

```
for (tx = stcol, i = x;
    !(col_hidden[tx] && ((i -= fwidth[tx]) < 0)); /* negazione di C */
    tx++) {
    /* corpo vuoto */
}
```

La negazione di C si può semplificare applicando le leggi di De Morgan⁶, ottenendo

```
for (tx = stcol, i = x;
    col_hidden[tx] || ((i -= fwidth[tx]) >= 0);
    tx++) {
    /* corpo vuoto */
}
```

Osserviamo che il ciclo `for` scritto in questo modo non è molto comprensibile: si inizializzano due variabili anziché una, e la condizione si riferisce indirettamente alle due variabili del ciclo. In base a quanto detto nella Sezione 2.3 potrebbe essere preferibile riscrivere il ciclo usando un costrutto `while` anziché `for`:

```
tx = stcol, i = x;
while (col_hidden[tx] || ((i -= fwidth[tx]) >= 0)) {
    tx++;
}
```

3.6 Conversione di un carattere numerico in un intero

Si vuole realizzare una funzione `int ctoi(char c)` che dato un carattere c che deve appartenere all'insieme `'0', ... '9'`, restituisce il valore intero corrispondente. La soluzione seguente è chiaramente inadeguata, perché consiste in codice “copia e incolla”:

```
int ctoi(char c)
{
    if (c == '0') {
        return 0;
    }
    if (c == '1') {
        return 1;
    }
    if (c == '2') {
```

⁶ Le leggi di De Morgan affermano che $!(P \ \&\& \ Q) == (!P \ || \ !Q)$, e $!(P \ || \ Q) == (!P \ \&\& \ !Q)$. In altre parole, la negazione di un *and* è l'*or* della negazione dei termini, mentre la negazione di un *or* è l'*and* della negazione dei termini.

```

        return 2;
    }
    if (c == '3') {
        return 3;
    }

    /* ...eccetera... */

    if (c == '9') {
        return 9;
    }
    return 99;
}

```

È possibile semplificare la funzione ricordando che i caratteri '0', ... '9' hanno codici ASCII consecutivi: in particolare, '0' $\rightarrow$ 48, ... '9' $\rightarrow$ 57. Il valore numerico dei caratteri si può quindi calcolare come $(c - '0')$, che equivale a $(c - 48)$. Per assicurarsi che l'input sia valido è opportuno usare una asserzione (sezione 2.8): nel caso in cui l'asserzione non sia vera, il programma si interrompe immediatamente con un messaggio d'errore, mentre la funzione precedente ritorna un valore spurio (99) che probabilmente causerà problemi in un punto diverso del codice. La funzione può quindi essere riscritta come:

```

int ctoi(char c)
{
    assert( (c >= '0') && (c <= '9') );
    return (c - '0');
}

```


Bibliografia

Brian W. Kernighan, Dennis M. Ritchie, *The C Programming Language*, 2nd edition, Prentice Hall, Inc., 1988. ISBN 0-13-110362-8.

Brian W. Kernighan, Rob Pike, *The Practice of Programming*, Addison-Wesley, 1999, ISBN 978-0201615869.

Brian W. Kernighan, P. J. Plauger, *The Elements of Programming Style*, 2nd edition, McGraw-Hill, 1978, ISBN 0-07-034207-5.

David Gries, *The science of programming*, Springer-Verlag, 1987, ISBN 978-0-387-96480-5.

Steve McConnell, *Code Complete*, 2nd edition, Microsoft Press; 2004, ISBN 978-0735619678.

William Strunk, E. B. White, *The Elements of Style*, 4th ed., Pearson, 1999, ISBN 978-0-205-30902-3.